

Embedded Systems with the STM32F091RC

Sam Shue

August 2026

Table of contents

Preface	1
How this book is organized	1
Relationship to the MSP430 edition	2
 I. Foundations	 3
1. Introduction	5
1.1. Learning Objectives	5
1.2. What is an Embedded System?	5
1.3. Embedded Systems and Microcontrollers	6
1.4. Microcontrollers vs. Microprocessors	7
1.5. Why Embed a Computer?	10
1.6. The Economics of Embedded Systems	11
1.6.1. Is It Really Necessary to Embed a Computer?	11
1.6.2. Example: Digital vs. Analog	14
1.7. Limitations of Embedded Systems	18
1.8. Developing for Embedded Systems	19
1.9. Recap	22
1.10. References	22
 2. C Programming	 23
2.1. Learning Objectives	23
2.2. Programming embedded systems	23
2.3. Why Program in C?	27
2.3.1. Portent Portables	28

Table of contents

2.3.2.	Why Not Program in C++?	28
2.3.3.	Beyond C – Other Languages in Embedded Systems	29
2.4.	Introduction to C	30
2.5.	Creating Variables	31
2.5.1.	Operators	33
2.6.	Branching and Loops	35
2.6.1.	If Conditions	35
2.6.2.	While Loops	36
2.6.3.	For Loops	37
2.7.	Functions	38
2.7.1.	Scope	39
2.7.2.	Function Prototypes	41
2.8.	Arrays and Structs	42
2.8.1.	Arrays	42
2.8.2.	Structs	43
2.8.3.	The Typedef Keyword	45
2.8.4.	Strings	46
2.9.	Variable Sizes in Embedded Systems	47
2.9.1.	Modifiers and Qualifiers	48
2.9.2.	Integer Sizes Across Microcontrollers	51
2.10.	C Programming Conventions	52
2.10.1.	Avoiding “Magic” Numbers	53
2.11.	Variables and Memory	54
2.11.1.	Memory Map	54
2.11.2.	Advanced Considerations: Memory Behavior	58
2.12.	Pointers	62
2.12.1.	Pointers and Functions	65
2.12.2.	Pointers and Arrays	66
2.12.3.	Dynamic Memory Allocation in C	69
2.13.	C Build Process	72
2.13.1.	Preprocessor	72
2.13.2.	The Compiler	74
2.13.3.	The Assembler	76
2.13.4.	The Linker	76

2.13.5. The Loader	77
2.13.6. Cross Compilers	78
2.14. Integrated Development Environments	78
2.14.1. STM32CubeIDE	79
2.15. Recap	83
 II. Digital I/O and Program Structure	 85
 3. General Purpose Input/Output (GPIO)	 87
3.1. Learning Objectives	87
3.2. Interfacing with the Outside World	87
3.3. Microcontroller Documentation	89
3.3.1. Development Board Documentation	90
3.3.2. Microcontroller Family Documentation	90
3.3.3. Microcontroller-Specific Documentation	90
3.4. General Purpose Input Output	94
3.4.1. How to Find Operating Voltage	94
3.4.2. Controlling an LED	94
3.4.3. Reading a Button Press	97
3.5. Peripherals and the Memory Map	98
3.5.1. The STM32F091 Memory Map	101
3.6. GPIO Organization	101
3.6.1. Which Pins are GPIO?	101
3.6.2. Dev Board Pinout	103
3.7. Controlling the GPIO	103
3.8. Example Application: Blink an LED	107
3.8.1. Driving an LED with GPIO	107
3.8.2. Enabling the Peripheral Clock	108
3.8.3. Setting the Mode	108
3.8.4. Output Register	110
3.8.5. The Bit Set/Reset Register	110
3.8.6. Accessing Registers	111
3.8.7. Finding the Register Address	113

Table of contents

3.8.8. The “volatile” Keyword	114
3.8.9. Putting it Together	114
3.9. Example Application: Read a Button Press	115
3.9.1. Reading a Button Press with GPIO	115
3.9.2. Configuring the GPIO as an Input	116
3.9.3. Reading a Voltage	117
3.9.4. Do We Need a Pull-Up Resistor?	117
3.9.5. Internal Pull-Up and Pull-Down Resistors	118
3.9.6. Putting it Together	119
3.10. GPIO Hardware	121
3.11. Register Macros	122
3.12. Bit Masking	124
3.12.1. Understanding the Problem	125
3.12.2. Creating a Bit Mask	125
3.12.3. Bit Masking for Setting and Clearing Bits	126
3.12.4. Creating Readable Masks with the Shift Operator	127
3.12.5. Masking Registers with Paired Values	127
3.13. Additional Notes	128
3.13.1. Polling for Inputs	128
3.13.2. Button Debouncing	128
3.14. Recap	129
4. Embedded Programming	131
 III. Peripherals	 133
5. Timers	135
6. Analog-to-Digital Conversion (ADC)	137
7. Serial Communication	139

Preface

This book is an introduction to embedded systems programming using the STMicroelectronics STM32F091RC microcontroller and the STM32 Nucleo-F091RC development board.

Draft status

This book is a work in progress. Most chapter content still needs to be written.

It is intended for readers who are new to embedded systems, and covers the fundamentals of microcontrollers, C programming for resource-constrained targets, general purpose input/output (GPIO), embedded programming patterns, timers, analog-to-digital conversion, and serial communication. Development is done using STM32CubeIDE, STMicroelectronics' free Eclipse-based IDE for STM32 microcontrollers.

How this book is organized

The book is split into parts, each grouping related chapters:

- **Foundations** — what an embedded system is, and the C programming concepts needed to work with one.
- **Digital I/O and Program Structure** — controlling GPIO pins and structuring larger embedded programs (hardware abstraction layers, state machines, interrupts).

Preface

- **Peripherals** — timers, analog-to-digital conversion (ADC), and serial communication.

Relationship to the MSP430 edition

This book follows the same chapter structure as the companion “Embedded Systems with the MSP430G2553” textbook, but is written independently for the STM32 platform rather than translated from it, since most chapter content (peripheral registers, memory maps, toolchain) is platform-specific. See that project’s README for background on why these are maintained as separate repositories rather than one.

Part I.

Foundations

1. Introduction

1.1. Learning Objectives

In this chapter we will learn:

- What an embedded system is
- What is a microcontroller
- Why embed a computer
- What constraints do embedded systems have
- What is a development board?

1.2. What is an Embedded System?

An embedded system refers to a digital computer that is... embedded into another system! Think about all the devices you interact with daily that have been “digitized” by the inclusion of a small computer integrated into them to provide some sort of control or automation. Did you make coffee this morning with a coffee maker that automatically poured the perfect amount of coffee for your cup? Did you drive to work in a car that had a “push-to-start” button? Did you wash your clothes this week with a washing machine that knew how long to run each wash cycle based on the load size? These are all examples of systems that can automate many tasks due to their embedded computer!

1. Introduction

On the other hand, what is not an embedded system? An embedded system is integrated into another device to provide some sort of control or automation. A computer that we use for general computing purposes, such as a laptop computer, would not be considered an embedded system.

A great rule of thumb for determining whether a computer is an embedded system or not, is to consider what is the purpose of the computer. If the device contains a computer, but is purchased or sold for some functionality other than computation, then it is an embedded system. If the device is purchased or sold because the product *is* a computer, then it is not an embedded system. For example, if you bought a coffee maker that has an embedded computer, do you care how much memory it has or how fast the processor is? However, when you bought your latest smartphone, did you exam the specifications of that device (e.g., processor speed, memory size, storage capacity)?

1.3. Embedded Systems and Microcontrollers

The subject of embedded systems is A **microcontroller** is a device that contains an entire computer on a single chip, making it very easy to integrate with other systems. How are we able to fit an entire computer onto a single chip? Given the perspective of viewing a computer as something powerful like a laptop or desktop, this concept might feel a little far-fetched, but keep in mind we don't need our coffee maker to run "photoshop".

Because of the limited needs of the application, the computer might not need very many resources, allowing it to be stripped of any excess memory and processing power and shrunk down to a very small size, perfect for integration. Many microcontrollers will have memory on the scale of Megabytes while your laptop is easily on the Gigabyte scale. Microcontrollers come in a variety of different specifications - and it is up to the engineer to select the one most appropriate for the system in which it's being embedded, based on processing power, memory, and input/output demands.

1.4. Microcontrollers vs. Microprocessors

Circuit board containing microcontrollers

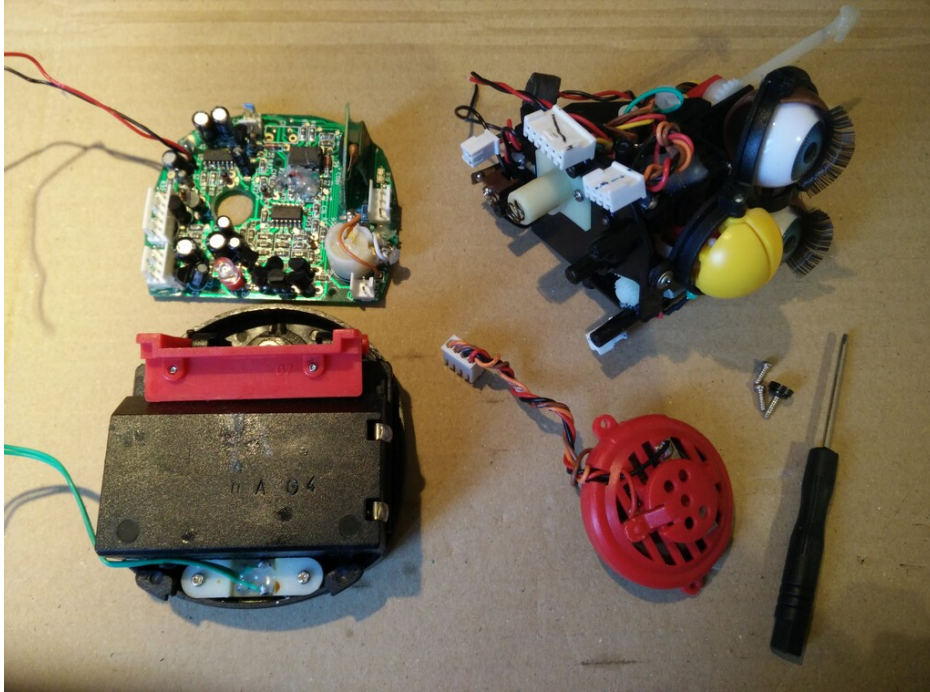


Figure 1.1.: Example microcontroller in embedded application (disassembled Furby electronic toy) [1]

1.4. Microcontrollers vs. Microprocessors

The term “microcontroller” may sound familiar to you, due to its similarity to the term “microprocessor”. A microprocessor is the “brain” of a computer, such as the AMD or Intel processors found in a laptop or desktop computer. Microprocessors are often quite powerful, especially when contrasted with the computational power of most microcontrollers, but they themselves are

1. Introduction

not an entire computer.

To understand what constitutes a computer, let's take a look at the five basic components of a Von-Neuman computer, which is the architecture of most modern computers:

In Figure 1.2, we can identify four major components:

- Memory
- Processor
 - Arithmetic Logic Unit
 - Control Unit
- Input
- Output

The memory is responsible for holding the computer's information, such as the instructions comprising the programs and any data currently in use. The arithmetic logic unit (ALU) is capable of performing computations on the data, such as addition, subtraction, etc. The control unit reads instructions and directs the ALU to perform the appropriate action. Together, the control unit and the ALU form the processor. The input collects data from the outside, such as through a keyboard or mouse, and the output can display information, such as a monitor, or perhaps even manipulate something, like printing a page on a printer.

The ALU and control unit are often bundled together and called the "processor". These two components are what comprise the microprocessor as well! As we can see here, the microprocessor is only two out of five components necessary for making an entire computer. The microcontroller actually contains all five of these components, but often in a more limited capacity than your laptop.

1.4. Microcontrollers vs. Microprocessors

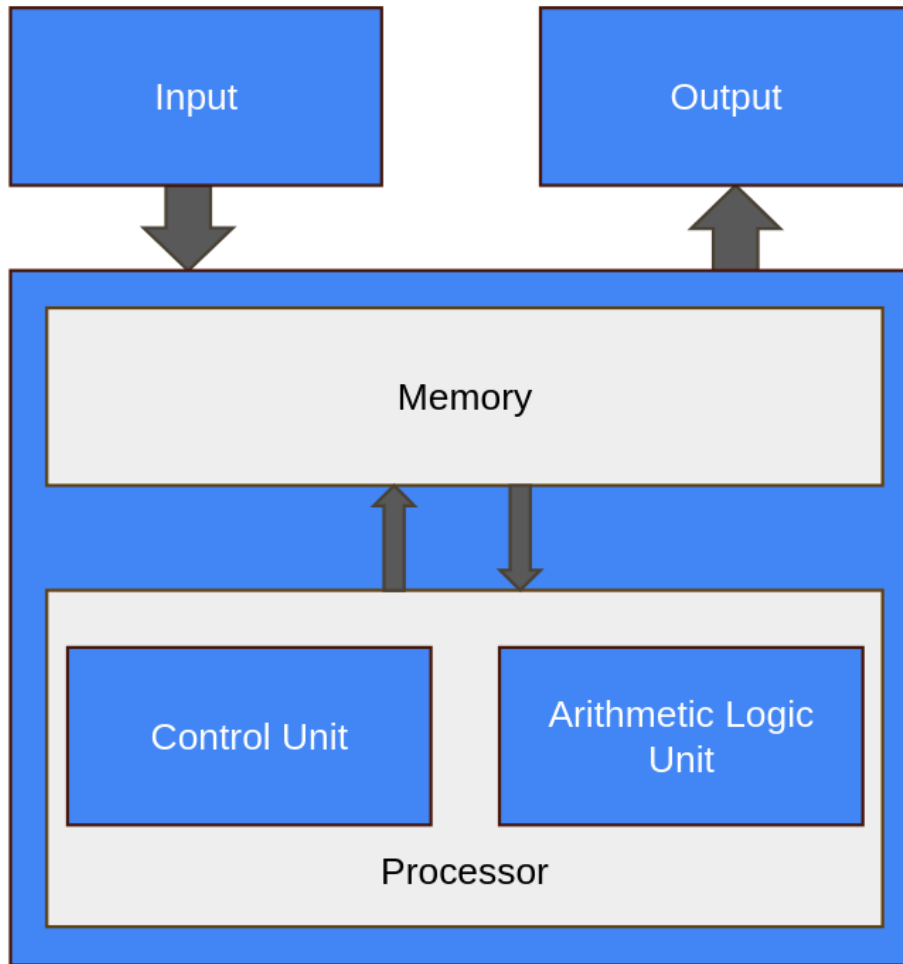


Figure 1.2.: Basic Structure of Von Neumann Computer

1. Introduction

1.5. Why Embed a Computer?

Embedding a computer into another system can bring many advantages. Since computers can be programmed to create a variety of behaviors, they can replace the need for engineers to design a custom circuit to perform specific tasks. Say an engineer is tasked with creating a circuit that will blink a warning light on a car's dashboard with a one second interval. This would usually require a custom circuit with a specific timer and clock frequency, but rather than design this circuit, the computer can be integrated and programmed to blink this light. Not only would it be capable of handling this task, but many others at the same time. This is a simple example, but microcontrollers can also imitate the behavior of much more complex circuits as well.

Embedding computer to avoid having engineers create custom circuits cuts down on a product's non-recurring engineering (NRE) cost. This is the cost of paying an engineer to create a new custom design. The engineer will still have to program the microcontroller, but this is generally easier than building and testing a custom-designed circuit.

Microcontrollers also help reduce the cost of a design because they are a generic component. The same microcontroller can be integrated into many different systems and be programmed for very different tasks. Microcontrollers will be manufactured on a scale that makes them extremely affordable, often costing less than 25 cent per chip. Even though the microcontroller's design may be far more complex than a custom circuit for a specific task, its mass manufacturing still makes it the more cost-effective option.

Microcontrollers can also bring more advantages than just expense. In the same space any other circuit would occupy, a microcontroller can implement additional capabilities. This could be additional features beyond the base functionality of the device, such as safety features – both for the operator and the electronics themselves. This is because we are implementing

features through software rather than hardware, which doesn't require more physical space!

1.6. The Economics of Embedded Systems

Now that we understand what an embedded system and a microcontroller are, let's take a closer look at why cost is such a large driver behind the widespread use of microcontrollers in modern products.

1.6.1. Is It Really Necessary to Embed a Computer?

In many embedded systems applications, an entire computer is used in place of what could otherwise be a custom analog circuit. One example of a device that utilizes a microcontroller for cost benefits and flexibility is a thermostat.

Traditionally, thermostats used analog circuits to control the temperature, employing components like resistors, capacitors, and relays. Figure 1.3 shows an example of one such analog thermostat control board. This board reads the temperature settings from switches or potentiometers set by the installer or homeowner, and switches relays on and off to turn a furnace or air conditioner on and off – all without a single line of code.

Many modern thermostats now utilize microcontrollers for enhanced functionality and cost-effectiveness, as shown in Figure 1.4.

Flexibility

- A microcontroller allows for programmability, enabling the thermostat to have various features, such as scheduling different temperatures throughout the day or displaying the current temperature on a digital display.

1. Introduction

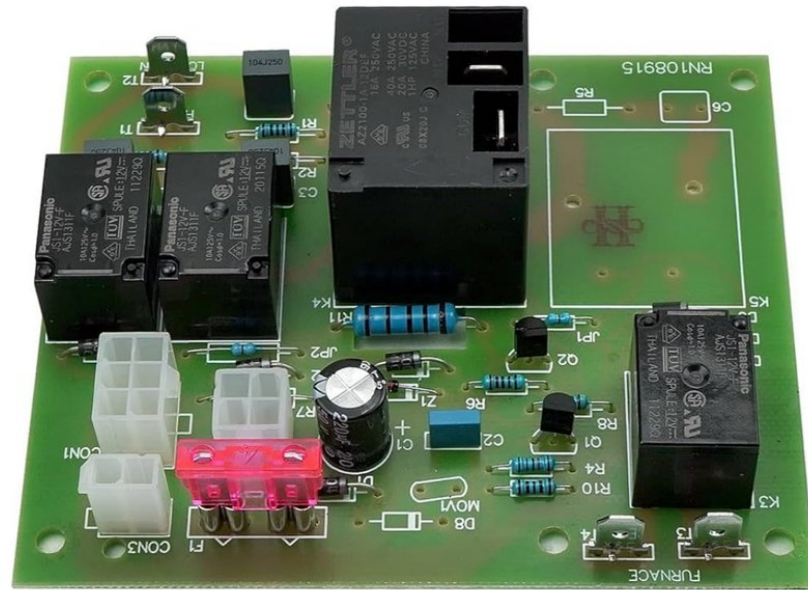


Figure 1.3.: An analog thermostat control board. Relays (the black components) and other discrete components are wired together to control heating and cooling equipment.

1.6. The Economics of Embedded Systems



Figure 1.4.: A microcontroller-based thermostat control board. The microcontroller (the black IC to the right of the 7-segment displays) reads button presses, drives the display, and switches the same kind of relay used in the analog design.

1. Introduction

- These features can be updated or customized through firmware updates, without having to redesign the analog control circuitry.

Cost-effectiveness

- Versatility and mass production benefits of microcontrollers often result in lower overall costs.
- Microcontrollers can handle multiple functions, reducing the need for additional discrete components. Instead of designing separate custom circuits for scheduling, button input, and display functionality, a single microcontroller can be programmed to handle all of these behaviors.

Diagnostic capabilities

- Microcontrollers can monitor various parameters and detect faults or inefficiencies in the system.
- This allows for proactive maintenance and troubleshooting, such as alerting a homeowner that a filter needs to be changed or that a sensor is reading an unexpected value.

Notice that both thermostat designs in Figure 1.3 and Figure 1.4 still rely on the same kind of relay to physically switch the furnace or air conditioner on and off. The microcontroller does not replace this hardware, but rather replaces the analog logic that used to decide *when* to activate it, while also adding capabilities – such as scheduling, a digital display, and diagnostics – that would be very difficult to implement using discrete analog components alone.

1.6.2. Example: Digital vs. Analog

Let's consider a concrete example: building a circuit that blinks an LED. We'll compare two circuits that accomplish this task – one built from

1.6. The Economics of Embedded Systems

traditional analog/digital components, and one that uses an entire computer (a microcontroller) for this simple task.

Figure 1.5 shows a design that blinks an LED using only a relay, a capacitor, and a couple of resistors – no microcontroller required. Pricing each of these discrete components individually gives us the following:

Component	Description	Price
Capacitor	1000 μ F, 16V, radial electrolytic	\$0.54
Relay	DPDT, 2A, 5V	\$1.91
Resistor	10 k Ω , 1%, 1/4 W	\$0.10
Resistor	330 Ω , 5%, 1/2 W	\$0.10
LED	Red, diffused, T-1 3/4	\$0.34
Total		\$2.99

This total excludes jumper wires, a breadboard, and batteries, and would likely be cheaper if these components were ordered in mass quantities.

Now let's consider the same task using a microcontroller, as shown in Figure 1.6. Pricing these components individually gives us the following:

Component	Description	Price
LED	Red, diffused, T-1 3/4	\$0.34
Resistor	20 Ω , 1%, 1/4 W	\$0.10
Microcontroller	16-bit MCU, 2 KB Flash, DIP-14 package	\$1.89
Total		\$2.33

Even though the microcontroller design replaces several discrete components (the relay and capacitor used to create the timing behavior) with an entire computer, the total cost is actually *lower* than the analog design (\$2.33 vs. \$2.99). This is because the microcontroller, despite being a far more complex device internally, is manufactured at an enormous scale across

1. Introduction

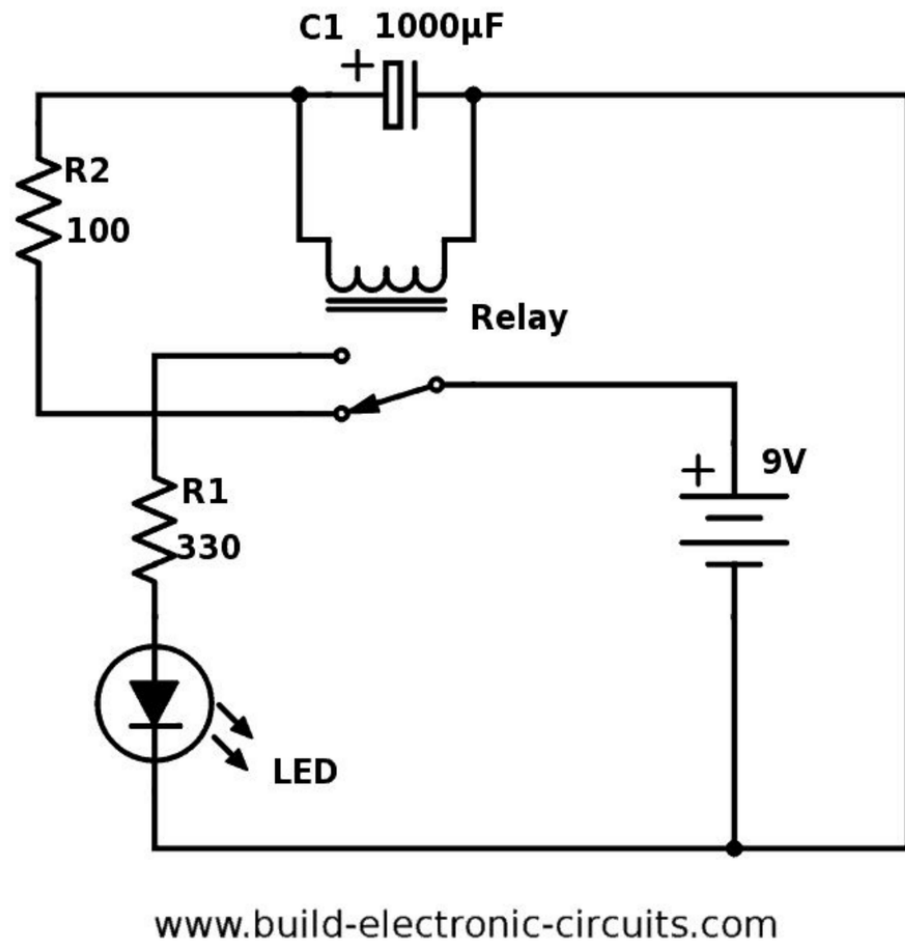


Figure 1.5.: Blink an LED using a relay, capacitor, and resistors.

1.6. *The Economics of Embedded Systems*

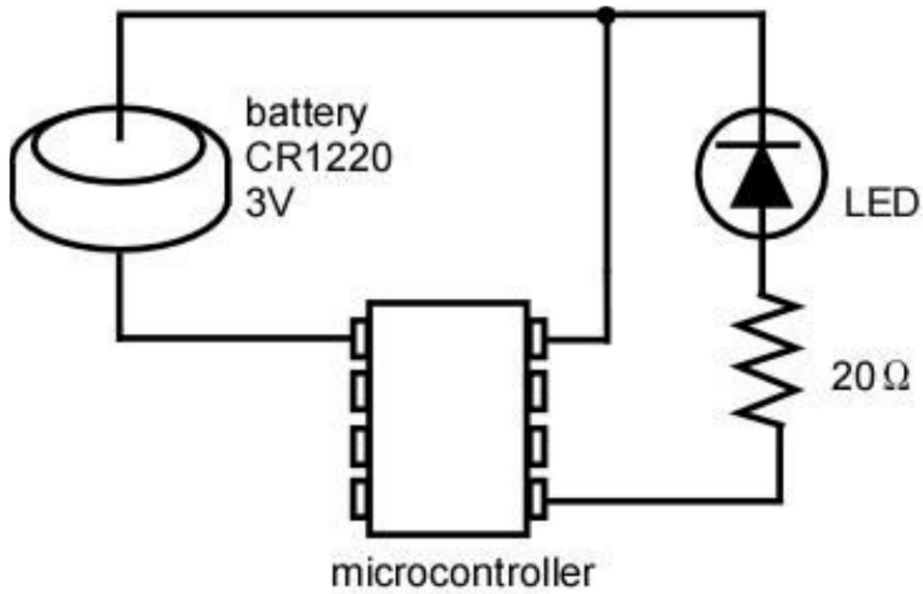


Figure 1.6.: Blink an LED using a microcontroller and a resistor.

1. Introduction

many different products and applications, driving its unit cost down below that of the specialized analog components it replaces. Like the analog design, this total also excludes a breadboard, jumper wires, and batteries, and would likely change if either design were purchased in mass quantities.

1.7. Limitations of Embedded Systems

While we have observed many benefits that can come from embedded systems, such as cost reductions and automation, there are also many limitations that must be considered when designing for them. It was mentioned earlier that embedded systems are not as powerful as the microprocessors found in your laptop computer, and therefore their computing power and other resources are limited. This must be kept in mind when selecting the embedded system appropriate for the application.

If limited resources are such a concern, why not use a microcontroller with more memory and processing power than what's required for the application? There are two reasons for this: cost and power consumption. Cost is the more obvious of the two – more resources, which equals more hardware, which equals higher cost. This cost may only increase the microcontroller's cost by a few cents, but when the product that is utilizing the microcontroller is being manufactured on the scale of thousands to millions of units, these pennies begin to add up. Power consumption is another major concern. More memory means more hardware components that require power, and faster processors have a higher clock frequency, which means more transistors switching, which means higher power consumption. Embedded systems often find themselves at home in battery-operated mobile devices, therefore keeping the power consumption low is of upmost importance to ensure battery life is elongated as much as possible. Have you ever purchased one device, such as a smartphone or laptop, over another because one had a longer battery life?

1.8. Developing for Embedded Systems

Embedded systems are also sometimes limited by their input and output capabilities. Most embedded systems come with some features that allow them to produce digital signals, read analog voltages, and communicate with other devices using serial protocols. These features can vary from microcontroller to microcontroller, so one must be selected that contains the necessary interfacing hardware, cost, and computational resources. It is the embedded systems engineer's responsibility to select the correct microcontroller based on the project's demands for these criteria.

1.8. Developing for Embedded Systems

When developing for an embedded system, the design must first go through a prototyping phase to verify functionality. This prototyping phase usually involves breadboarding the circuit that will integrate the selected microcontroller, and the microcontroller will often be interfaced through a **development board**.

Development boards are prototyping boards that contain a microcontroller and onboard tools and components that aid in development and testing. These components often include the following:

- Voltage regulation
- Programmer circuit
- Debugger
- Breakout header pins
- Push buttons and LEDs

These components often appear on development boards because they are commonly needed by programmers to power the board, upload code, and test inputs and outputs.

1. Introduction

Let's look at an example development board, the STM32 Nucleo-F091RC. This board has a lot going on, but what is most important is the chip at the center of the board, the STM32F091RC. This is the microcontroller, and all the other components here are to help with programming and testing of this chip.

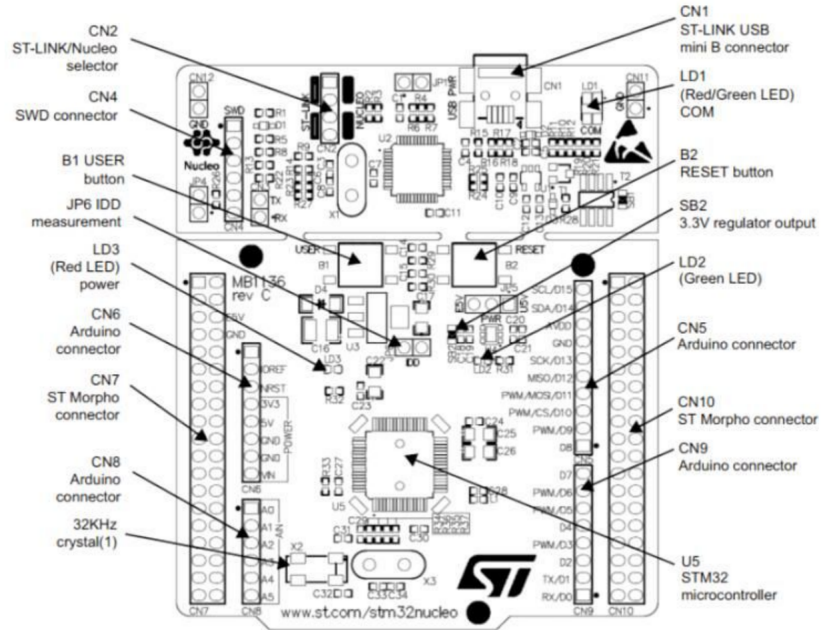


Figure 1.7.: An overview of the STM32 Nucleo-64 development board. (Source: STMicroelectronics, UM1724 STM32 Nucleo-64 boards user manual.)

The top half of the board contains the “ST-LINK” debugger circuitry. A debugger allows us to meticulously examine our microcontroller’s code behavior by setting breakpoints and stepping through the code line by line, while simultaneously being able to check memory contents. If this

1.8. Developing for Embedded Systems

doesn't make much sense right now, don't worry, we will be elaborating on this again later in the book. This portion of the board also provides power regulation, which provides power to the microcontroller at a specific voltage. This part of the board also provides a virtual serial port connection from the microcontroller to your laptop or desktop computer, allowing communication between the two.

On the lower half of the board, beyond the jumper isolation block, we have the socket for the microcontroller, along with some extra connected components.

Arguably, the most important addition here is the inclusion of "breakout" pins. Breakout pins are pins that are large and spaced out, designed to be easily connected to jumper wires for interfacing with a prototyping circuit. The pins on a typical microcontroller package are much too small to attempt to plug or solder wires directly into, so the development board contains traces (wires embedded into the board) that connect these tiny package pins to the breakout pins.

In addition to the breakout pins, development board very commonly include components such as buttons and LEDs, as they are very often used in prototyping scenarios for easy getting user input via a button press or indicating the status of something through illuminating an LED.

Separating the top "half" from the bottom, we see a "jumper isolation block", labeled CN2 (the "ST-LINK/Nucleo selector") in Figure 1.7. These jumpers bridge electrical connections between the debugger circuit and the microcontroller. These bridged connections allow power and communications to reach the lower part. However, if we were planning on someday building a custom circuit using the STM32F091RC, we wouldn't be including a debugger/programmer, since we wouldn't expect the person purchasing the embedded product, the "end user", to reprogram the device. These jumpers allow us to disconnect the microcontroller and test the chip without the support of that circuit, emulating what it might behave like before we create a custom board.

1. Introduction

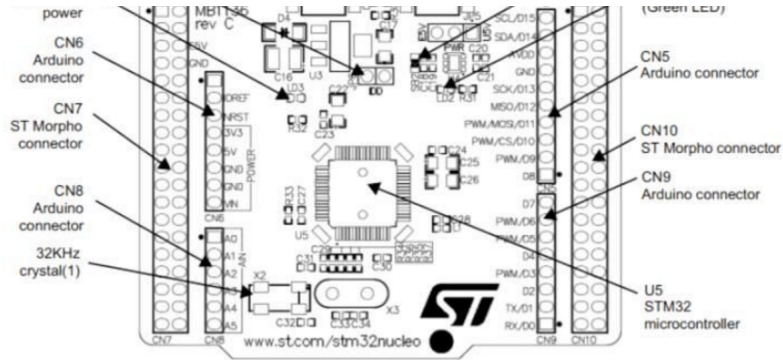


Figure 1.8.: The STM32F091RC microcontroller (U5) and the surrounding Arduino-compatible (CN5, CN6, CN8, CN9) and ST Morpho (CN7, CN10) breakout headers that give access to its pins. (Source: STMicroelectronics, UM1724 STM32 Nucleo-64 boards user manual.)

1.9. Recap

An embedded system is an application-specific computer system which is built into a larger system or device. Using a computer system enables improved performance, more functions and features, lower cost, and greater dependability. With embedded computer systems, manufacturers can add sophisticated control and monitoring to devices and other systems, while leveraging the low-cost microcontrollers running custom software.

1.10. References

1. P.-L. Ageneau, “Chaplier Fou,” Aug. 25, 2023. [Online]. Available: <https://chapelierfou.org/blog/a-usb-furby.html>

2. C Programming

2.1. Learning Objectives

This chapter will serve as a C programming refresher, or a to-the-point introduction for those who have programmed in other languages but are using C for the first time. In this chapter we will provide the motivation for using C, the basic syntax, how C works with computer memory, and how to use the STM32CubeIDE to develop C programs for STM32 microcontrollers.

In this chapter we will learn:

- The differences between low- and high-level languages
- Assembly language and machine code
- C Programming basics
- C and memory management

2.2. Programming embedded systems

Like any computer, embedded systems must be programmed before they can do anything. Before we get into how we program them, let's step back and define just what a program is. Because we use computers every day, I'm sure we all have an intuitive idea what a program is. What modern operating systems have come to refer to as an "App" may come to

2. C Programming

mind, or maybe you think of a video game, or a text editor like Microsoft Word. While these are all programs, a program by definition is much more simplistic than any of these. A program is a set of instructions stored in memory to be executed by the processor.

This definition begs the question, what is an instruction? An instruction is some simple operation that a computer can do. This can be an arithmetic operation, like add two numbers together; something logistical like compare two numbers; or even skip some other instructions based on a condition. Each of these operations is defined by the processor's **instruction set architecture** (ISA). The ISA specifies a list of all operations the processor is capable of and how those instructions are formatted.

Let's take a look at an example of one of these instructions.

```
ADDS R5, R5, #10
```

This instruction adds 10 to the value stored in a memory location R5 and saves the result back to that location. R5 refers to a memory location inside a small memory component called the register file. This instruction could be used to compute a simple mathematical expression, such as

```
x = x + 10
```

Where x would be stored in the register file memory location, R5. We will see that when writing programs, we will often refer to the computer's memory components, such as the aforementioned register file. While memory can have many intricacies, for now let's just understand that memory in general has many locations for holding data, all indexed by a number that we call the **address**. In this example, R5 represents the address of a memory location (location 5) in the register file.

These instructions are presented in a way that we, as the programmers, can understand them. Hence why instructions, such as "ADD", are presented in English. Naturally, they are translated into **machine code**, or the binary

Address	Data
0x00000000 →	0b0001 0101
0x00000001 →	0b1101 0001
0x00000002 →	0b0001 0111
0x00000003 →	0b0101 0100
0x00000004 →	0b0001 0100
0x00000005 →	0b0001 0101
0x00000006 →	0b1101 0001
0x00000007 →	0b0001 0111
0x00000008 →	0b0101 0100
0x00000009 →	0b0001 0100

Figure 2.1.: An example of computer memory addressing

2. C Programming

1's and 0's that a computer understands by the use of a program called a **compiler**. The language that we just saw is what we call **assembly language**, which directly corresponds to the machine code produced by the compiler.

In our previous example, we were able to add a number (10) to an existing number (the data stored in R5). Let's consider another example, but this time let's specify a mathematical operation we want our computer to perform:

```
x = x + 10 + 15 + 20 + 25
```

Now we want to sum up a variety of numbers. How would we write a program that does this? Assuming that x corresponds to register file location R5, we could write:

```
ADDS R5, R5, #10  
ADDS R5, R5, #15  
ADDS R5, R5, #20  
ADDS R5, R5, #25
```

And this will perform this operation. This series of instructions, or program, starts from the top instruction and performs each operation sequentially. Computers perform one instruction after the other, and we can see by the time we reach the last line, we would have computed the result of the equation described earlier. One observation that can be made even this early, is as the application's complexity increases, so do the number of instructions. You can imagine that writing in assembly language would be very laborious for programmers creating complex applications such as Microsoft Windows. To assist with this problem, other programming languages have been created, such as C.

To implement the equation described earlier in C, we would simply type:

2.3. Why Program in C?

```
x = x + 10 + 15 + 20 + 25;
```

Which looks nearly identical to the equation itself! While the computer in the end would have to perform the same operations we saw in assembly, describing this in C is much simpler on the programmer's part. The compiler will then be responsible for interpreting this line of C code into the associated assembly instructions, and from there into the computer's native machine code. Because of the added step of going to assembly and then to machine code, we classify C as a “**high-level**” programming language. High-level refers to how distanced the lines of code are from the machine code instructions that would be required to execute them. In contrast, assembly language would be considered a “**low-level**” language.

2.3. Why Program in C?

This question might seem obvious at first, simply because the complexity of C seems lesser compared to our earlier example, which is largely still true. The one drawback to using C and other high-level languages is the efficiency. We've now taken full control of writing the individual instructions that comprise our program and handed it over to the compiler. A compiler may not be as skilled at interpreting the high level instructions into low level instructions, resulting in a less efficient and larger program.

This is the trade-off that exists with any high-level language: the further from the individual instruction the language is, the less efficient it tends to be. However, this trade off is nearly worth it every single time, as it would be too challenging to write modern programs directly in assembly. Not only would getting these programs written would be challenging, but the number of bugs (accidental, unintentional program behaviors) as a result of the complexity would be great.

2. C Programming

2.3.1. Portent Portables

Writing programs in C does bring with it some other advantages, most notably: portability. We've mentioned assembly language and ISA, but we did not mention that each type of processor has its own specific ISA, and therefore a different set of instructions that they are capable of executing. The assembly language understood by the processor in your laptop computer is likely very different from the one in your smartphone. Therefore, programs written in assembly for one machine cannot be moved and executed on a different machine that has a different ISA. The C language however must be interpreted into assembly by the compiler before being interpreted to machine code. Therefore, if a compiler exists for each target machine, the C code can be rebuilt for each machine. This makes the C language more generic, and therefore more portable.

2.3.2. Why Not Program in C++?

If you're already familiar with programming, you may be familiar with the C++ language. C++ is a language derived from C, adding in many new features used by programmers to organize code. These features include a concept called a "class", which allows programmers to group related functions and data together, making it what we refer to as an **object-oriented** language.

While these features are attractive from the programmer's point of view, they also cause the language to be even further distanced from the instruction level than C. Because of this, C++ programs don't compile as efficiently as most C programs do. This typically is too much of a tradeoff, as microcontrollers still have limited amounts of memory, and cannot store very large programs. C++ is gaining traction in the world of embedded systems development, especially as many modern microcontrollers have more memory than ever before, but still C rules supreme. There are also historical reasons for this as well, as many older projects active in both

2.3. *Why Program in C?*

hobbyist and industry spaces, are still ongoing and originally written in C.

2.3.3. **Beyond C – Other Languages in Embedded Systems**

While C remains dominant, other languages are gaining interest for embedded development:

- **Rust**
 - Memory-safe without a garbage collector
 - Prevents many bugs common in C (e.g., buffer overflows)
 - Still emerging in embedded space due to:
 - * Limited ecosystem and tooling
 - * Steeper learning curve
 - * Smaller community and fewer drivers/libraries
- **Python / MicroPython / CircuitPython**
 - Easy to learn and use
 - Great for rapid prototyping and educational use
 - Limited to higher-end MCUs due to:
 - * Large interpreter overhead
 - * Slow performance and high memory use

2. C Programming

2.4. Introduction to C

In this section we'll get familiar with the basic syntax of the C language. A language's **syntax** refers to the structure and keywords that exist within that language. Keywords or reserved words are words that are recognized by the compiler that have some explicit meaning or function. Let's begin by analyzing an example of a simple C program:

```
#include <stdio.h>
int main(){
    // print hello world to terminal
    printf("Hello World!");
    return 0;
}
```

This is a simple program that prints the message, “Hello World!”, to the standard output, which if ran on a traditional laptop or desktop computer, would be some terminal application.

<show hello world terminal screenshot>

The first thing to point out about any C application is that the program begins with the **main** function. We'll define functions in C more formally later on, but for right now, know that lines of code that follow main and are contained within the curly braces are the instructions that come next.

In this example, we have only two lines of C code. One is another function call, “printf()”, and the other is “return 0”. An individual line of C code is easy to identify, because each line ends with a semicolon. This tells the compiler when a single line of a C program is finished and ready to be interpreted into the resulting assembly instructions. The “printf” function will write to the output whatever text is placed within the parenthesis that follow it. The return keyword ends the current execution of instructions within the function, therefore terminating our program and passing a 0

2.5. Creating Variables

back to the operating system that called it. This 0 is typically a code that indicates that everything executed okay.

At the very top of the code we see an additional line, which reads: `"#include <stdio.h>"`. This is an example of a preprocessor macro. The **preprocessor** is a part of the compilation process that runs before the compiler does. It processes any lines that begin with a `"#"` symbol and in this case is used to retrieve additional code to add to our program. This additional code is referred to as a **library**, and it gives us access to the `"printf"` function. Libraries are a handy way to provide other programmers prebuilt functionality, so you do not have to re-write everything from scratch.

We also skipped over the line that began with `"//"`. The double backslashes indicate a comment. A comment is a note left to add context or a message. These are usually left to remind the programmer what a complex series of instructions might be doing, or to explain something to another programmer who also is developing in this code base but didn't write a certain portion of the code. Comments may also encapsulate multiple lines using the `"/*"` and `"*/"` syntax:

```
/*  
Comments here  
Comment here  
And here. All on different lines!  
*/
```

Comments are ignored by the compiler and do not add to the overall code size.

2.5. Creating Variables

If we wish to write our own programs, we need to be able to store data. Like many other programming languages, C uses variables to achieve

2. C Programming

this. Variables look just like a variable in a mathematical expression, and function in a similar way. A variable represents a location in the computer's memory that has been reserved to hold data.

In C, variables must be given a type. The type determines what kind of data will be stored in that variable. Let's look at an example of how one might declare a variable:

```
int main(){  
    int x;  
    return 0;  
}
```

Note

This declaration simply reserves space in memory for the variable “x”; it does not assign it an initial value.

In this example, we've declared a variable called “x” and it has been designated to hold an integer by preceding our variable name with the keyword “int”. There are many different data types in C, but we'll examine some of the basic types here:

Data Type	Description
int	holds only integer numbers
float	holds decimal point numbers
char	used to hold values that correspond to characters in the ASCII table
bool	used to hold true or false values. Short for “Boolean”

These variables can all be used in different scenarios depending on the need of the data type.

2.5.1. Operators

Now that we're able to reserve memory locations to hold our data in the form of a variable, we need operators to be able to actually do things with the data. There are many operators in C, but we'll start with the most simple, the assignment operator, "=", which allows us to write data to a variable. Let's modify our earlier example to assign a value of 5 to our variable `x` we declared:

```
int main(){
    int x = 5;
    return 0;
}
```

Now after reserving the space for variable `x`, the value 5 will be written to that location. There are many more operators available in C, so let's have a look at all the different types available.

i Note

If a variable is not assigned a value, the value it contains is undefined – it could be any leftover bit pattern that happened to already be at that memory location.

2.5.1.1. Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations, such as addition, subtractions, etc. These operations require variables or values to perform the operation with, and a destination variable to write the result. Let's see an example of the addition operator:

2. C Programming

```
int main(){  
    int x = 5;  
    int r = x + 10;  
    return 0;  
}
```

We've expanded our earlier example by adding an additional variable, `r`, which hold the result of our addition operation, which adds 10 to the value stored in `x`, 5. After this operation, `r` will hold the value, 15. There are many arithmetic operators in C, here are some of the most common:

Operator	Description
+	performs addition
-	performs subtraction
*	performs multiplication
/	performs division

2.5.1.2. Boolean Operators

In addition to arithmetic operators, we also have a set of Boolean operators. Boolean operators are operations that do not have a numerical result, but rather a true/false result. Boolean operators are used to compare two numbers for properties such as equality, greater than, etc.

Operator	Description
<code>x == y</code>	used to compare two values for equality
<code>x != y</code>	used to compare two values for inequality
<code>x > y</code>	used to evaluate greater than
<code>x < y</code>	used to evaluate less than

<side note: are adding variables (x,y) more or less confusing?>

2.6. Branching and Loops

Boolean operators are often used in conjunction with branching and loops within a C program. Branching refers to performing different actions based on a condition, and looping allows us to re-run lines of code over and over. These structures give us the ability to make decisions within a program.

2.6.1. If Conditions

“If” conditions are the essential branching method in C. An “If” condition will run a specified group of instructions based on a Boolean condition. Let’s take a look at an example:

```
int main(){  
    int x = 5;  
    int r = 0;  
    if( x > 2){  
        r = x + 10;  
    }  
    return 0;  
}
```

Here we see the keyword, “if”, followed by parenthesis. Within these parenthesis, a Boolean expression is described. If the Boolean expression resolves as true, the lines of code contained within the following parenthesis will be executed. Otherwise, it will be skipped and execution will continue.

If conditions can also be expanded upon by adding additional conditions and an “else” condition that resolves when any of the preceding conditions are not true. An example of this is demonstrated in the code below:

2. C Programming

```
int main(){
    int x = 5;
    int r = 0;
    if( x > 2){
        r = x + 10;
    } else if(x > 5) {
        r = x + 15;
    } else {
        x = r + 5;
    }
    return 0;
}
```

2.6.2. While Loops

In addition to conditional branches, programmers often have the need to repeat certain lines of code. The “while” loop serves this function by repeating grouped lines of code so long as its Boolean condition resolves as true:

```
int x = 0;
while ( x < 5){
    x = x + 1;
}
```

In this while loop, the body of the loop (the lines of code encapsulated by the curly braces) will execute five times, terminating once x reaches 5, as the condition will no longer evaluate as true. The condition of a while loop is evaluated prior to executing the body, so if x had already been greater than 5, the body of the loop would not execute even once.

If there is a need to execute a body of code once, THEN check the condition, a do-while loop may be used:

2.6. Branching and Loops

```
int x = 5;
do {
    x = x + 1;
} while ( x < 5);
```

Here, the body of the loop will be ran BEFORE checking the condition.

2.6.3. For Loops

While all of our looping needs can be satisfied by the while loop, another type of loop exists as a shortcut for common looping conditions: the for loop. It is very common that programmers might want a loop to execute for a certain number of iterations. To achieve this, a while loop can be constructed and a counting variable might be used, similar to how the variable `x` behaved in our while loop example earlier. A for loop incorporates a counting variable into its syntax, allowing easy control over how many times a loop iterates.

```
int i;
for(i = 0; i < n; i++){
    // body of the loop goes here
}
```

The for loop header is composed of three parts: initialization, condition, and increment/decrement. We can see here that in the parenthesis that follow the keyword “for”, the variable `i` is initialized to 0. This is followed by the condition, `i < n`, which like in the while loop, tells the body when to stop looping. The last part is the increment/decrement, where the variable is incremented by one (`++` is the c shortcut for adding 1 to a variable). Each part is separated by a semicolon.

2. C Programming

2.7. Functions

Functions are the main source of organization in the C language. Functions allow us to group lines of code and invoke them at various points in our program. Let's consider an example and discuss the syntax:

```
float circleArea(float radius){  
    float area;  
    area = radius * radius * 3.14;  
    return area;  
}
```

This is a function that computes the area of a circle given a radius value. The beginning of the function has three parts: the return value, the name, and the parameters.

The return value is what the result of the function is; for a circle that computes the area of a circle, we expect a decimal point value as the result, so our return value is simply indicated by a “float”.

The function name is “circleArea”. This part is not a keyword, and can be named whatever we want. We've chosen a name that is indicative of the function's purpose.

The parameters are the values that are passed to the function. Functions can have one, many, or no values passed to them. The parameters are contained within the parenthesis and contain a data type and a variable name. The values passed to a function are copied from their original locations, and these variables will only exist between the curly braces that contain the body of the function.

Now that we've discussed the format of a function, we can revisit our main function we saw in the “Hello World” example:


```
int main(){
    // print hello world to terminal
    printf("Hello World!");
    return 0;
}
```

We can see how `main` follows the same structure as our other function example. We'll note that `main` takes 0 parameters and returns an integer. The `main` function is called by the operating system if on a desktop or laptop computer, and the integer it returns encodes an error value. While functions can be named anything, `main` is an exception, as the compiled program will start at the function called `main`.

2.7.1. Scope

As mentioned before, variables declared within a function are not accessible outside of that function, and they only exist in memory so long as that function is still be executed. This limited availability of a variable is what we refer to as the variable's "scope".

Let's consider an example using the `area` function earlier:

```
float circleArea(float radius){
    float area;
    area = radius * radius* 3.14;
    return area;
}
int main(){
    float r = 5;
    float area = circleArea(r);
    ...
}
```

2. C Programming

Here, the variables “r” and “area” within “main” are only accessible within the main function’s curly braces. The function “circleArea” is called within main, so main’s variables do not get removed from memory yet, but are not accessible within the “circleArea” function. We can observe that circleArea has two variables within its scope, “area” and “radius”. The “radius” variable is populated with a copy of the data contained in “r” when the function is called. We also notice that our variable “area” shares a name with the “area” variable from main. Variables must have unique names so long as they are in the same scope, but here, since these “area” variables are only available within their respective functions, this isn’t a problem and they are both easily differentiated from each other due to their scope. This kind of scope is what we call a **local scope**, as their scopes are local to the functions where they were declared.

We can also bypass the local scope of a variable by declaring a variable outside of the functions.

```
float area = 0;
void circleArea(float radius){
    area = radius * radius* 3.14;
}
int main(){
    float r = 5;
    area = circleArea(r);
    ...
}
```

Here, “area” has been moved and declared outside of both functions, giving it a **global scope**. Global variables are scoped to all functions that exist within the same C source code file. By making “area” global, we no longer have to create multiple local variables and return the area from the function.

This may seem like a convenient shortcut, but when projects become large, this scales poorly and overuse of global variables can result in wasted memory and very difficult to trace bugs.

2.7.2. Function Prototypes

In the examples we just saw, all of our functions were defined above the main function, and this was no coincident. In C, source files are compiled from the top-down, which means if a function is called before the compiler has processed that the function exists (“circleArea”, for example), it won’t be able to perform the compilation, and will throw an error. As files grow larger, programmers often wish to place main() closer to the top of the file, given the importance of main and understanding how a program behaves. However, we need some way to do this without breaking compilation.

Function prototypes exist to remedy this issue. A function prototype is a line of code that alerts the compiler of the existence of a function, what parameters it takes, and what it returns; but excludes the implementation. Let’s use our example from earlier, but instead we’ll place main() above circleArea()’s implementation:

```
#include <stdio.h>
float circleArea(float radius);
int main(){
    float r = 5;
    float area = circleArea(r);
    printf("Area: %f", area);
    return 0;
}
float circleArea(float radius){
    float area;
    area = radius * radius * 3.14;
    return area;
}
```

Now circleArea() has a prototype that resides at the top of the file, and the implementation of the function exists below main. This prototype will be encountered by the compiler before the call to it in main(). When

2. C Programming

a function call is compiled, the compiler doesn't need to know what the function does, just what needs to be passed to and returned from the function. Since the compiler knows what "circleArea()" is, it can finish the compilation, just so long as it eventually finds an implementation that pairs with the function prototype. If it doesn't, then the code will fail to compile.

2.8. Arrays and Structs

We've discussed the basic data types of the C language, but often we need more than a single value to represent different concepts or a collection of values.

2.8.1. Arrays

Arrays are used to maintain a contiguous sequence of data in memory.

Let's look at an example of an array:

```
int a[5] = {4, 6, 2, 8, 0};
```

Here, the array is declared following the data type it holds like any other variable, but the defining line of syntax is the angle brackets that follow the name. Within the angle brackets the number of elements contained within the array is defined. Here, the compiler makes space in memory for five integers one after another. Following those brackets, the array has been initialized with some values to fill the five elements specified. These are contained within the curly braces and separated by commas. In this case, the "5" within the angle brackets could be removed, as the initialized list of elements is implicit of the size of the array.

Each element in "a" is accessed by an "index", which begins at 0. For example:

```
int r = a[0] + 10;
```

This code accesses the first element of the array which exists at index 0. The last element of the array exists at index 4. An array in C is a very simple structure, as the array itself only maintains the position in memory where the array begins, and just counts the requested index worth of space away from that starting position. Arrays have no way of knowing how large they are, as they are just given the requested amount of space when they are declared. Because of this, programmers must be careful, because the following code will compile with valid syntax for our current example:

```
a[9001] = 74;
```

This is clearly outside of the number of elements the compiler created for us, but to the computer it just wants to access the memory location that is 9001 integers away from the start of the array. This can result in the computer attempting to read from an invalid memory address.

2.8.2. Structs

Structs are a way to group related variables in C. For instance, we might want to maintain an x, y point in 2-D space. I could declare two separate float variables for x and y, but then I need to keep track of them together throughout the rest of the code, and it might not be immediately clear to other programmers that they are a pair. To resolve this issue, a struct can be used:

```
struct point {  
    float x;  
    float y;  
};
```

2. C Programming

This defines a structure called point, and this can be used as if it were a new datatype. For instance, I can declare an instance of my struct and assign values to x and y as follows:

```
struct point p;  
p.x = 2;  
p.y = 3;
```

We can see here that I declared the struct using “struct point” followed by a name for the variable, “p”. I then accessed and assigned the members of that struct, x and y, using the “.” Syntax. This operator allows me to access the variable contained within the specified structure. The variables within a struct can be used like any other variable after being accessed through the “.” operator.

Here is a more complete example, using a struct to represent a 2-D point and compute the distance between two points:

```
#include <stdio.h>  
#include <math.h>  
// Define a struct for a 2D point  
struct Point {  
    int x;  
    int y;  
};  
int main() {  
    // Declare two points  
    struct Point p1, p2;  
    // Initialize the first point  
    p1.x = 3;  
    p1.y = 4;  
    // Initialize the second point  
    p2.x = -1;  
    p2.y = 2;
```

```

// Calculate the distance between the two points
double distance = sqrt((p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y));
// Print the coordinates of the points and the distance between them
printf("First Point: (%d, %d)\n", p1.x, p1.y);
printf("Second Point: (%d, %d)\n", p2.x, p2.y);
printf("Distance between the points: %.2f\n", distance);
return 0;
}

```

2.8.3. The Typedef Keyword

Having to precede every instance of our struct's name with the "struct" keyword can be tedious. To avoid repeating "struct" every time we declare a variable of this type, we can use the **typedef** keyword to define our struct as a new data type.

typedef is used to specify a new name for a data type. It requires the following pattern:

```
typedef <thing that is a new type> <name of new type>;
```

We can apply this to our point struct by combining the definition of the struct itself with a typedef, giving the resulting type a name directly:

```

#include <stdio.h>
#include <math.h>
// Define a struct for a 2D point
typedef struct {
    int x;
    int y;
} Point;
int main() {

```

2. C Programming

```
// Declare two points
Point p1, p2;
// Initialize the first point
p1.x = 3;
p1.y = 4;
// Initialize the second point
p2.x = -1;
p2.y = 2;
// Calculate the distance between the two points
double distance = sqrt((p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y));
// Print the coordinates of the points and the distance between them
printf("First Point: (%d, %d)\n", p1.x, p1.y);
printf("Second Point: (%d, %d)\n", p2.x, p2.y);
printf("Distance between the points: %.2f\n", distance);
return 0;
}
```

Notice that we can now declare variables of type “Point” directly, without needing to write “struct Point” each time.

2.8.4. Strings

When writing programs, especially any that display information to a user, we are interested in storing entire words and sentences. Earlier we introduced the `char` data type, which was used to store individual characters.

```
char c = 'A';
```

However, we need to be able to store a group of characters to form words or sentences. This is where the concept of a string comes in. A **string** refers to an array of characters ending with a null character.

2.9. Variable Sizes in Embedded Systems

```
char s[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

This array is a string that holds the word “Hello”. Note that the last element contains ‘\0’, which represents the null character. A null character is used to indicate the end of a string, and is essential when attempting to determine where a string ends, as arrays do not know how long they are. By including a null character at the end, we can create loops that iterate through a string and stop when they encounter ‘\0’ as that indicates the end of the string has been reached.

2.9. Variable Sizes in Embedded Systems

One question that might come to mind is why differentiate between data types? If you think about it, you should be able to do everything with a “float”. Decimal point numbers can represent integers, we could use them to hold corresponding character values as well, and even use them to represent true and false by designating a certain value to represent “true”. The answer comes down to speed and efficiency. Binary can naturally represent integer values, but it requires a special format to represent decimal numbers. Floating point numbers are also difficult to process and require special hardware to be processed efficiently. Floats also take up more memory than chars or bools, and depending on the machine, “int”s as well. Therefore, we define other data types for faster processing and better use of the limited memory available on our microprocessor.

<Add example here of two programs producing variables of different sizes>

To this end, C defines even more varieties of data types that give use even more control over the amount of memory reserved for each data type.

2. C Programming

Data Type	Description
short	an integer that uses even less memory space than int, but holds smaller numbers
long	an integer that uses more memory space than int, but holds larger numbers
double	a decimal point number that uses more memory than float, but holds larger numbers

2.9.1. Modifiers and Qualifiers

We also might want to be able to further control the data types and how they're used, and to aid with this, C allows us to specify modifiers and qualifiers for our data types.

2.9.1.1. Modifiers

Some common modifiers are:

Modifier	Description
long	increases the memory allocated to a data type, allowing it to hold larger values
short	decreases the memory allocated to a data type, allowing it to hold smaller values while using less memory

2.9. Variable Sizes in Embedded Systems

Modifier	Description
<code>unsigned</code>	treats a variable as holding only non-negative values, doubling the range of positive values it can represent
<code>signed</code>	treats a variable as being able to hold both positive and negative values (the default for most data types)

These modifiers can be used in conjunction with most basic data types. For instance, I can declare:

```
long int x;
```

Which makes an integer that on most desktop machines holds 64 bits rather than 32 bits. I can also use the “short” modifier to make a 16-bit integer. Other modifiers can determine whether a variable is treated as having negative values or not:

```
unsigned int x;
```

This prevents “x” from being treated as though it has a negative value. From a numerical perspective, this allows a greater range of positive values, as we no longer have to dedicate the sign bit to representing negatives. This modifier is used a lot in microcontroller programming, as we are often modifying data that exists in memory locations where each bit’s value is important, and the numerical value of the data isn’t of interest. This will be contextualized in the following chapter.

2. C Programming

2.9.1.2. Qualifiers

C also supports type qualifiers. Qualifiers specify how the variable can be used within a program. Some common qualifiers include:

Qualifier	Description
<code>const</code>	specifies that a variable's value is constant and cannot be changed after it is assigned
<code>static</code>	allows a variable to retain its value between function calls, even after going out of scope
<code>volatile</code>	forces the compiler to build every instruction involving the variable, preventing it from being optimized out

The `const` qualifier specifies the value of a variable is constant. This means after the variable has been assigned a value, it cannot be changed. This is useful for defining actual mathematical constants in our program, such as pi:

```
const float PI = 3.14;
```

While you could hard-code 3.14 everywhere in memory where you wanted to use pi, this enhances the readability of the code. It can also be useful for values that serve as thresholds that may need to be adjusted throughout a program's development.

The `static` keyword allows a variable to hold its value even after going out of scope. This allows functions to essentially have their own variable that retains its memory between function calls.

2.9. Variable Sizes in Embedded Systems

```
int increment(){  
    static int x = 0;  
    x = x + 1;  
    return x;  
}
```

In this example, the first call to the function will return 1, then the following call will return 2, as the initialization of the static variable, “x”, only occurs on the first call, and it retains its data between subsequent calls.

The volatile qualifier is especially relevant for embedded systems as we will see in coming chapters. The volatile keyword forces the compiler to explicitly build all instructions that the variable is involved in. Sometimes the compiler will attempt to analyze and optimize C code as it translates it into assembly, and because of this process, it may exclude some operations that it evaluates as unnecessary. The volatile keyword prevents this from happening, and forces all of its operations to be built.

2.9.2. Integer Sizes Across Microcontrollers

Sometimes, a project’s requirements may change during development, which may demand the developers select a different microcontroller with more appropriate features. Since we’re programming in C, this isn’t as laborious of a process as if we were programming in assembly, in which case the program would have to entire re-written for the new microntroller! However, there are still some discrepancies that may still affect a language like C that provides us greater portability. This discrepancy is the standard `int` datatype size.

On most desktop computers, the C compiler defines that a standard `int` is 32 bits. However, for embedded C compilers, this may be inconsistent. A compiler for a 16-bit TI MSP430 might define an `int` as being 16 bits, while hte compiler for a 32-bit STM32 ARM compiler might define it as

2. C Programming

32-bits. If the embedded programmer wants to be mindful about memory consumption and is carefully selecting the integers based on their size, this can be a problem when moving a project's C code from one microcontroller architecture to another.

To resolve this issue, most embedded C compilers define a library called `stdint.h`. This library provides custom integer data type definitions that will preserve the integer size across architectures.

Integer Type Name	Description
<code>uint8_t</code>	8 bit unsigned integer
<code>uint16_t</code>	16 bit unsigned integer
<code>uint32_t</code>	32 bit unsigned integer
<code>uint64_t</code>	64 bit unsigned integer
<code>int8_t</code>	8 bit signed integer
<code>int16_t</code>	16 bit signed integer
<code>int32_t</code>	32 bit signed integer
<code>int64_t</code>	64 bit signed integer

When writing C code for embedded systems, it is considered best practice to utilize the standard integer data types rather than the native `int` for consistency.

2.10. C Programming Conventions

As programs grow larger and are worked on by more than one person, following good conventions becomes just as important as understanding the syntax itself. One common convention worth adopting early is avoiding “magic” numbers.

2.10.1. Avoiding “Magic” Numbers

A “magic” number is any number that is hard coded into a line of code with no context and is specific to that application. This could be something like a threshold value or a count value.

While these numbers are necessary, you can avoid making them “magic” numbers by either using a `#define` macro or a `const` variable:

- This will provide a name for the number, which will make it easy to read and provide context.
- If the number needs to change, it will only need to be changed where it is defined, rather than in every line of code using the number.
- These numbers should be formatted in `ALL_CAPS_SNAKE_CASE`.

Consider this C program that prints the area of a circle to the terminal:

```
#include <stdio.h>
int main(void){
    // Compute area of circle and print to terminal
    printf("area: %f", 3.14 * 2.0 * 2.0);
    return 0;
}
```

The values 3.14 and 2.0 are magic numbers – a reader has no way of knowing at a glance that 3.14 represents pi or that 2.0 represents a radius. Removing the magic numbers gives us:

```
#include <stdio.h>
#define RADIUS 2.0
#define PI      3.14
int main(void){
    // Compute area of circle and print to terminal
```

2. C Programming

```
printf("area: %f", PI*RADIUS*RADIUS);  
return 0;  
}
```

This version is both easier to read and easier to maintain, since the radius or the precision of pi used only need to be changed in one place.

2.11. Variables and Memory

When programming microcontrollers in C, it is especially important for the programmer to be aware of how their code is affecting the use of memory, especially since this tends to be a limited resource. Depending on where a variable is declared within the lifetime of a program, it may be stored in a different location and exist for either a limited amount of time or for the duration of the program.

2.11.1. Memory Map

Memory in a computer is generally divided up into different regions, each with a specific purpose. diagram below shows the most common memory regions. The diagram below shows these regions. The order and address ranges of these regions may differ between microcontroller architectures, or even include additional memory regions, but across all architectures the ones presented here will be present.

These regions can be separated into two categories: **Read Only Memory (ROM)** and **Random Access Memory (RAM)**. Let's take a look at the purpose of each of these regions and their respective category.

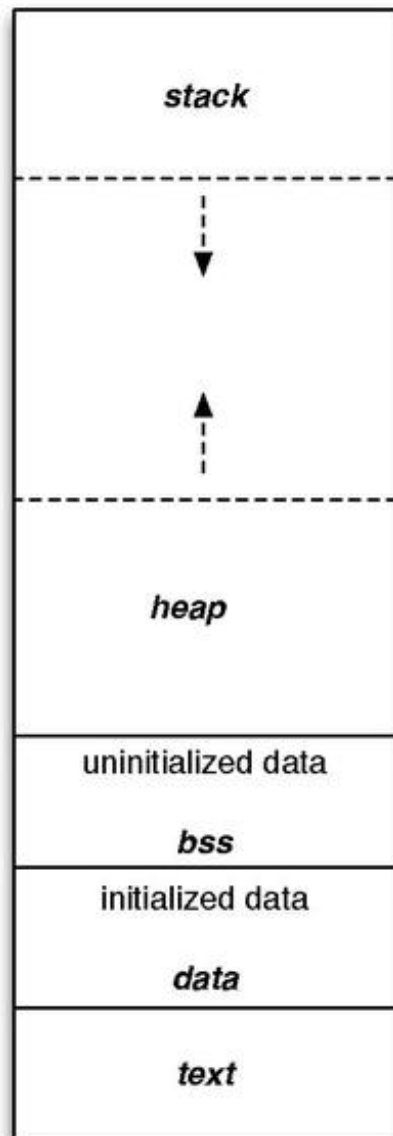


Figure 2.2.: Generic Microcontroller Memory Map

2. C Programming

2.11.1.1. Text

The region labeled text is where the instructions from the compiled program are stored. Once a program has been compiled and loaded, these instructions cannot be changed, making this region ROM.

2.11.1.2. Data and BSS

The data and BSS regions hold global, static, and constant variables. If a variable is uninitialized or initialized to 0, it is stored in the BSS regions. Otherwise, it gets stored in the data region. These regions can be manipulated at runtime, making it a form of RAM.

2.11.1.3. Stack

The stack is where local variables are stored, and it behaves in a first-in, last-out fashion, like a stack of items in real life. Local variables are automatically placed on the stack when they are created, giving them the name, “**automatic**” variables. In most architectures, the stack grows from larger address towards lower addresses. The nature of the stack means that it must exist in RAM.

2.11.1.4. Heap

The heap is where dynamically allocated variables are stored. These are variables created at runtime by other instructions and are not stored in the stack. The programmer determines the lifespan of these variables by having to be manually responsible for their creation and deletion. The heap grows in the opposite direction of the stack, to best utilize the range of memory allocated for both of these structures. Like the stack, this region is also considered RAM.

2.11. Variables and Memory

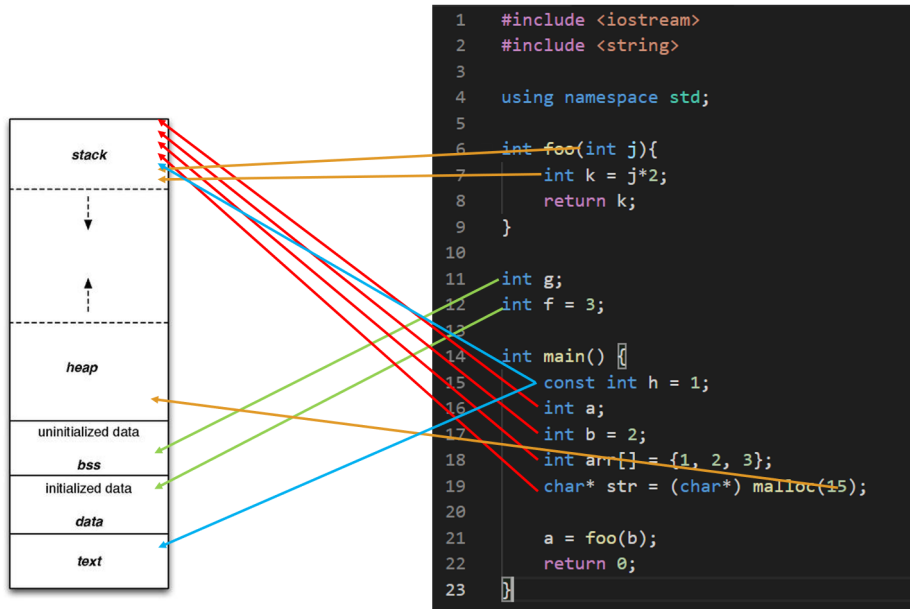


Figure 2.3.: Example C code and the associated memory location of each variable. Note that some compilers may optimize const variables during compilation and place them within the text region.

2. C Programming

2.11.2. Advanced Considerations: Memory Behavior

2.11.2.1. Instructions and the Text Region

The text region contains the individual instructions that comprise the compiled program. These instructions are the binary representation of the Arm Thumb instruction set used by the STM32F091RC (a Cortex-M0 microcontroller). Figure 2.4 shows how a compiled C line of code could be represented by a set of assembly instructions. Here, a single line of multiple addition operations would result in multiple assembly addition instructions.

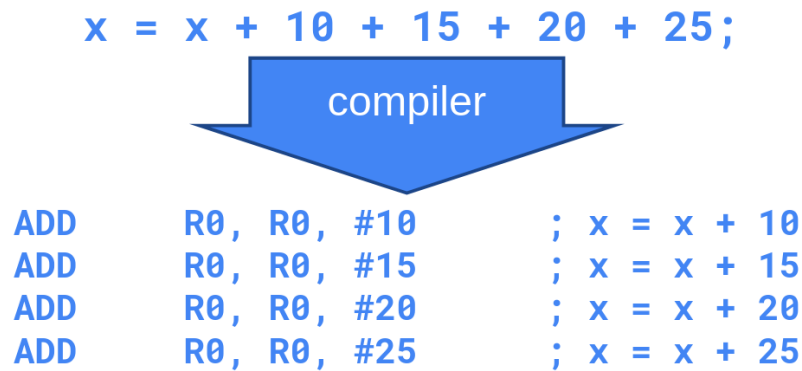


Figure 2.4.: Example section of the text region containing compiled instructions.

These instructions are then placed in the text region and read out and executed by the processor in a linear sequence. The processor has a special register called the “Program Counter” (PC) that tracks the memory location of the current instruction being read. As the program is executed, the program counter will increment to the memory location of the next instruction.

2.11. Variables and Memory

For example, to execute the example code in Figure 2.5, the program counter would start at address 0x08000100 to read out the instruction “ADDS R5, #10”. Since the current instruction requires 2 bytes (all Thumb instructions on the Cortex-M0 are 16 bits wide), the next instruction is 2 bytes away from the current instruction address, so 2 will be added to the program counter’s value. The program counter would then read out the instruction “ADDS R5, #15”. The process will then continue. The program counter will simply increment to the next instruction unless a branching condition, such as an if statement or a function call, would cause the program counter to jump to another location in code.

2.11.2.2. Stack Behavior

All variables that are declared inside of a function (with some exceptions, such as const and static variables), are stored within the stack. The stack functions in a way that is intuitive to its naming: variables are piled up on top of each other in memory – growing from high addresses to lower addresses. For a simple C program, we might see stack contents similar to the diagram in Figure 2.6.

Note that in Figure 2.6 the compiler has created space on the stack to hold all of the local variables within main. Since the code is paused before the call to the “foo” function, the contents of variable “y” are still uninitialized; only space on the stack has been allocated for “y” to exist – the contents of “y” are unknown at this time, as they are whatever bit values happened to be in that memory location prior to allocation of space for “y”.

We can also observe the memory contents for each variable in this figure. Note that “x” is allocated 4 memory locations, as integers require 32 bits on the STM32F091RC (an Arm Cortex-M0 processor, which is a 32-bit architecture), and memory is addressed in 8-bit segments. If we used 32 bits to represent 5 in binary, it would look like “0000 0000 0000 0000 0000 0000 0101”. But if we look at the memory location used to hold “x”, we see the least significant byte, “0000 0101” comes before the rest of the

2. C Programming

Instruction	Memory	Address
ADD R0, R0, #10	0x0A	0x0800011C
	0x00	0x0800011D
	0x00	0x0800011E
	0x08	0x0800011F
ADD R0, R0, #15	0x0F	0x08000120
	0x00	0x08000121
	0x00	0x08000122
	0x08	0x08000123
ADD R0, R0, #20	0x14	0x08000124
	0x00	0x08000125
	0x00	0x08000126
	0x08	0x08000127
ADD R0, R0, #25	0x19	0x08000128
	0x00	0x08000129
	0x00	0x0800012A
	0x08	0x0800012B

Figure 2.5.: Example section of the text region containing compiled instructions.

2.11. Variables and Memory

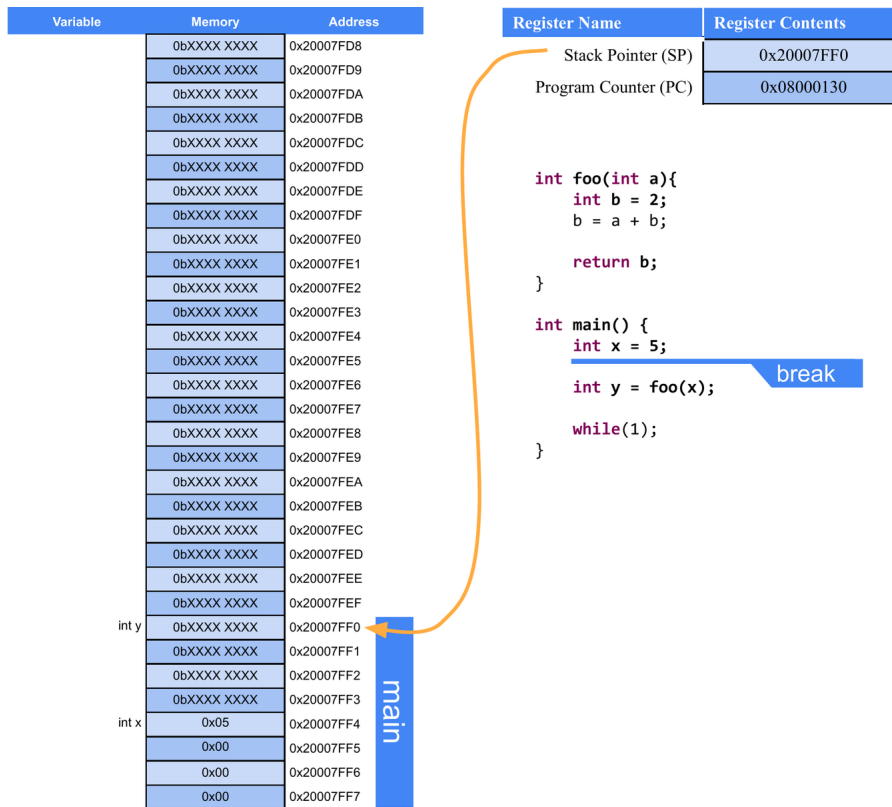


Figure 2.6.: Example C code and the condition of the stack up to the indicated break point. “X” indicates bit values that have been uninitialized.

2. C Programming

zero bytes. This is because the Arm architecture (as configured on the STM32) is “little endian”, which refers to the order that data is broken up into when stored in memory. “Little endian” means the least significant byte goes in the lowest memory address. This contrasts with “big endian”, which would store the bytes in reverse. The order bytes are stored in memory is architecture-specific, but ultimately has no consequence on the device performance and is an arbitrary decision.

Each function in C is given its own “stack frame” when called. We can see that in Figure 2.6 where a contiguous section in memory is carved out for all of “main’s” variables. If we let the program break at a point in “foo”, we’ll see another stack frame created for the “foo” function, as illustrated in Figure 2.7.

In this figure we can see the “foo” stack frame stacked on top of the “main” stack frame. Note that the stack frame of “foo” contains all of the local variables (including input parameters) along with a “return address”. The return address is used to indicate where in the instruction memory to return to once the “foo” function concludes. We can note between Figure 2.6 and Figure 2.7, the return address in Figure 2.7 is 4 bytes greater than the value of the Program Counter in Figure 2.6, which makes sense as the next instruction is a few bytes away from the instruction that called “foo”, and, once “foo” is complete, we should call the next instruction after “main” would have called “foo”.

2.12. Pointers

Pointers are one of the most confusing and unique aspects of both C and C++ programming. A pointer is a variable that holds the memory address of another variable, thus causing them to “point” to another variable. While confusing in nature, this allows specific control and access of memory locations, which we will see is very important for microcontroller programming. Many languages try to hide this aspect of programming

2.12. Pointers

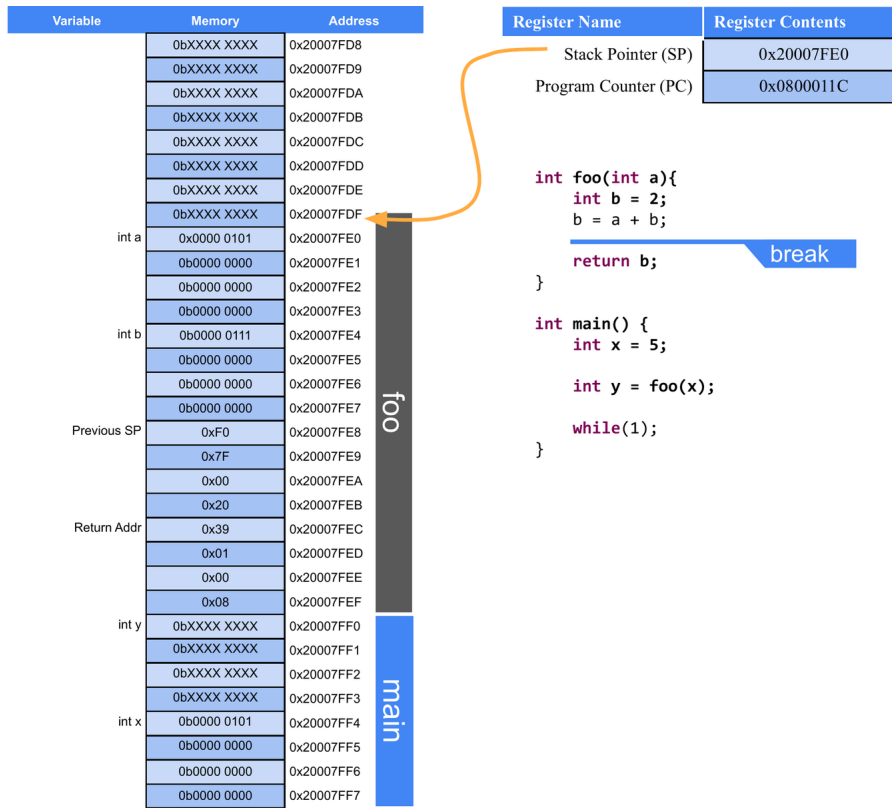


Figure 2.7.: Example C code and the condition of the stack up to the indicated break point.

2. C Programming

from the programmer, as mismanagement of pointers can lead to some very serious bugs.

Let's see an example of a pointer:

```
int* p;
```

From this, we can see that pointers are declared like any other variable. What makes the declaration different is the addition of the asterisk following the data type. This is what defined the variable as a pointer, by indicating what type of data the pointer will hold the address. In this case, variable “p” will hold the address of another integer variable.

So to actually use the pointer, we need another variable to point to.

```
int* p;  
int x = 5;  
p = &x;
```

Now we've added an integer variable x, and used a new operator, “&”, which extracts the address from the variable it precedes. This takes the address of x and stores it in p. Now, I can use “p” to manipulate “x” through the memory address it holds.

```
*p = 10;
```

This line of code assigns the value 10 to the variable “x”. This is done by use of the asterisk that precedes “p”. This is called “**dereferencing**” the pointer variable, which means that we follow the pointer back to the memory address it holds and use the variable that exists there. This allows us to both read and write that's variables data.

<Create visual reference for what's happening in memory here>

2.12.1. Pointers and Functions

So far we've introduced pointer syntax, but how is it useful? In C, one of the most valuable usages of a pointer is to allow other functions to utilize a variable that is out of scope. Instead of passing a variable to a function, which results in a copy being made, we can pass a pointer to variable, which allows a function to make permanent changes to a variable outside of its scope. For example, we can write:

```
void swap(int* x, int * y){
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

This function will permanently switch the values of the integers passed to this function as pointers.

```
int main(){
    int a = 1;
    int b = 2;
    swap(&a, &b);
}
```

In this example, we called swap by using “&” to get the addresses of “a” and “b” to pass to the function. The function dereferences those pointers to make changes to variables “a” and “b”, which aren't normally accessible by “swap”. After the swap function is called, the value of “a” is 2 and the value of “b” is 1.

```
void swap(int *x, int *y){
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

2. C Programming

```
}  
  
int main(void){  
    int a = 1;  
    int b = 2;  
    swap(&a, &b);  
    return 0;  
}
```

Before the call, the stack contains the variables **a** and **b** in **main**. During the function call, the addresses of those variables are passed to **swap**, which creates its own temporary value, **temp**, on the stack.

After the assignments complete, the original memory locations for **a** and **b** have been updated, even though the variables themselves are still defined in **main**. When the function returns and the main stack frame is restored, we can see that the variables in **main** have been swapped.

This is also why pointer parameters are so useful in embedded systems: they allow a function to modify data that is stored elsewhere in memory without needing to copy the whole variable. A pointer provides a direct route to the memory location that must be changed.

2.12.2. Pointers and Arrays

Arrays have a unique relationship with pointers as well. Earlier, we mentioned that arrays only retain the location in memory where the array begins. Because of this, we can use the name of the array as a pointer. For example:

```
int a[] = {1, 2, 3, 4}  
int* p = a;
```

2.12. Pointers

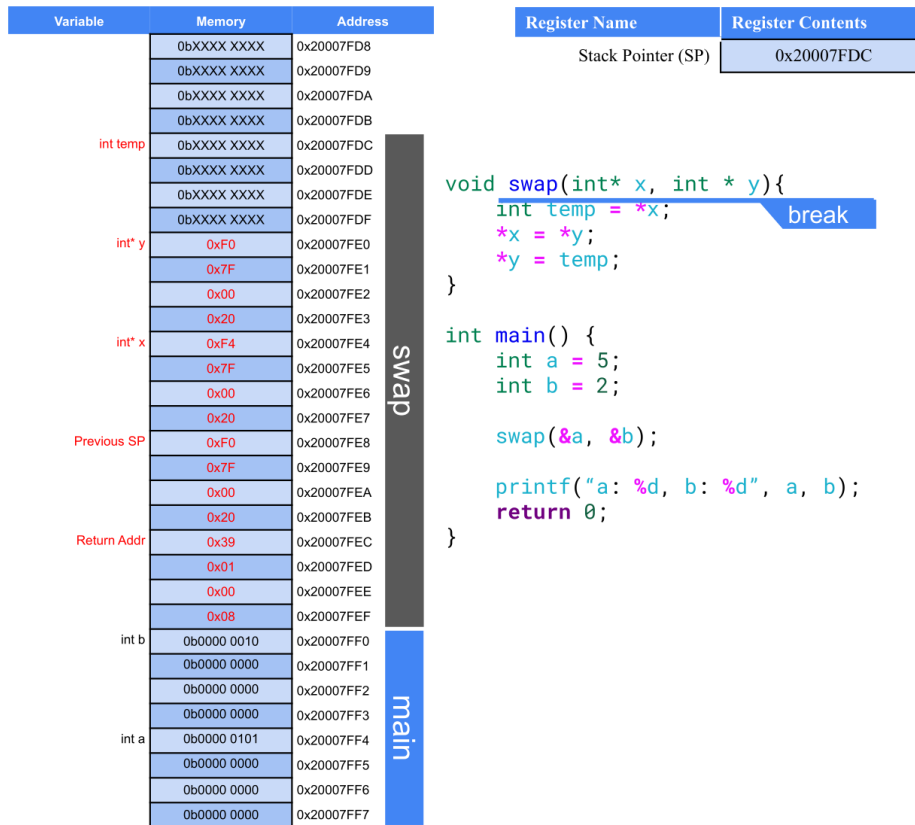


Figure 2.8.: Stack frame layout showing how a function call stores local variables and parameters in memory

2. C Programming

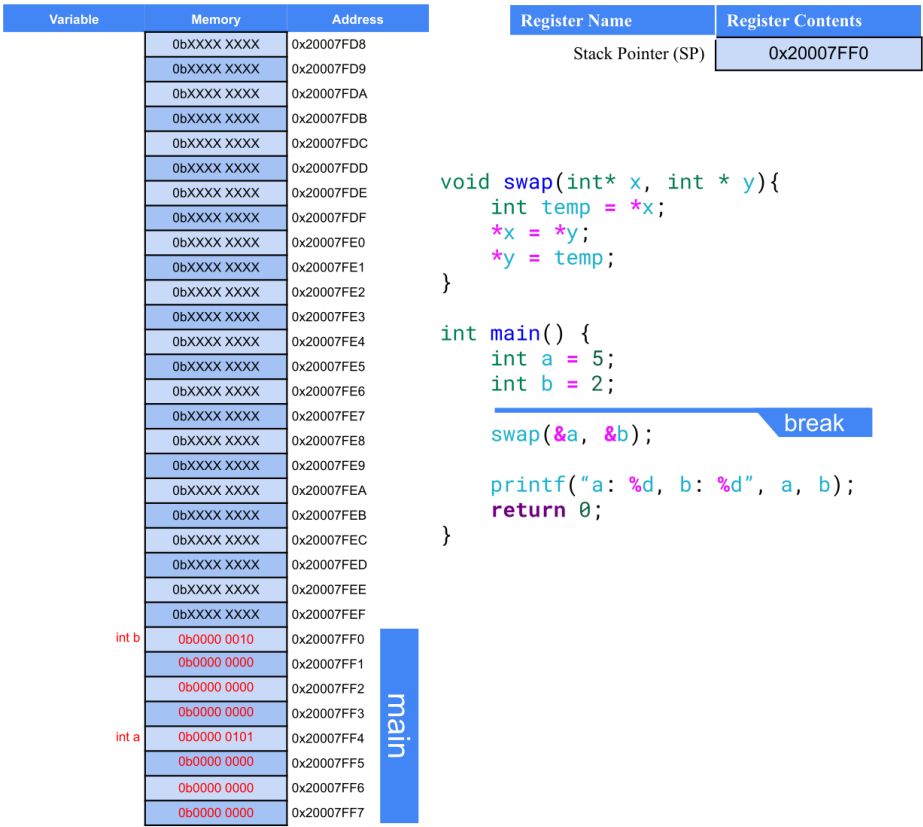


Figure 2.9.: The updated stack state after the swap call returns to main

Here, I can use “a” to assign integer pointer, “p”, as the name of the array refers to the address where the array begins. Because of this relationship, we can then pass pointers to arrays to other functions to modify the original location of an array, just as we did with other variables. The important thing to observe here is the absence of the & operator. Since the name of the array already refers to the address, we do not have to & prior to extract the address.

2.12.3. Dynamic Memory Allocation in C

Utilizing the heap in a C program can be a very tricky thing, and if handled improperly, can lead to memory leaks, which are very dangerous to an embedded system. Dynamic memory allocation refers to utilization of the heap to store data which will not be subject to the behavior of the stack. The stack is constantly growing and shrinking, and functions are called and completed. Sometimes we have a need to maintain data for a limited period of time but might need to exist for a time longer than the call of a function. This is where the heap comes in.

In the heap, we can specify that space for a particular data type, array, or structure may be allocated. The space is then made, and we are returned the address to that space. That space can then be used like any other variable; however, it is then our responsibility to deallocate that space when we are finished using it. If we do not deallocate it, then that memory will be locked out for the rest of the program’s life span. If we continue using the heap and never freeing up the memory when we are finished, this leads to a **memory leak**. If this process continues over time, the heap will grow so large that it consumes too much of the memory range it shares with the stack and will begin overwriting it, resulting in all sorts of undesirable behaviors.

2. C Programming

2.12.3.1. Malloc

To allocate space in the heap, we use the function “malloc”, which comes from the stdlib library. Malloc is short for memory allocation. This function will reserve a specified number of bytes in the heap and return a pointer to that space. We can then manipulate the data reserved by malloc by dereferencing the pointer it returns. Let’s look at an example:

```
char* p = malloc(1);
```

This call to malloc reserves a single byte, which is the size of a character on all architectures. Malloc returns a void pointer (a pointer that doesn’t point to any specific data type) and assign it to a character pointer, as a character pointer will point to a single byte. The type of pointer we assign the space to indicates how that space will be used as well.

It might be a bit strange to allocate space for a single character, so let’s use malloc to allocate space for an entire string containing 6 characters (including the null character):

```
char* s = malloc(6);
```

We can then populate this string like we would any other array. Recall that the name of an array can be used interchangeably with the name of an array:

```
s[0] = 'H';  
s[1] = 'e';  
s[2] = 'l';  
s[3] = 'l';  
s[4] = 'o';  
s[5] = '\\0';
```


This has been very straightforward so far when it comes to malloc, as we've just indicated how many bytes we need to allocate and get the pointer to that space. However, this has been simple because each char only requires one byte. For instance, a float would require 4 bytes worth of space, so if we were allocating space for an array of floats, we would have to allocate $4 * n$, where n would be the number of elements in the float array we wanted to create.

We could manually specify the number of bytes required, but this can lead to miscalculations and a lack of code portability as the data sizes may change between microcontroller C compilers. In order to write the most dependable malloc function call, we will utilize the sizeof function. Sizeof returns the number of bytes for the data type passed to it. Therefore, to use malloc to allocate a float array of size n we would write:

```
float* a = malloc(n * sizeof(float));
```

2.12.3.2. Free

After we are done using the space we created with malloc, it is then our responsibility to free that memory back up so that it may be allocated at a later time for different purposes. To do this, we simply call the function "free" and pass it the pointer to the allocated space. To free up our float array allocated previously, we would write:

```
free(a);
```

Now all data allocated in the heap at the address stored in "a" would be available to be reallocated. Failure to do so would result in potential memory leaks.

2. C Programming

2.13. C Build Process

Now we've discussed the basic syntax of the C language, let's look into how we convert that syntax into an executable that can be ran on a computer. We've already introduced the tool responsible for this conversion, the compiler. The compiler reads the C source files and ultimately converts it into an executable file which contains the 1's and 0's of the architecture's instructions. However, the Compiler is actually composed of many different applications, each with a specific responsibility. We've already introduced some of these, such as the preprocessor, but here we'll see where they all exist within the compilation process.

The four main steps of the compilation process are as follows:

1. Preprocessor
2. Compiler
3. Assembler
4. Linker

Let's take a look at each stage's responsibilities in detail.

2.13.1. Preprocessor

The preprocessor runs before any of the other steps and is responsible for all lines of code that are preceded with the “#” symbol. These lines of code are called “**macros**” and are eliminated and replaced by C syntax in some way before the preprocessor passes the processed source code to the compiler. Preprocessor macros can be used to add constant values to locations in the C source code by using commands such as “#define” or used to add entire header files through “#include”.

Let's consider an example of the preprocessor's effects on a C source file. Let's say I have the following C source files:

main.c

```
#include "circle.h"
int main(){
    float r = 3.5;
    float area = circleArea(r);
    return 0;
}
```

circle.h

```
#define PI 3.14
float circleArea(float r);
```

circle.c

```
#include "circle.h"
float circleArea(float r){
    return PI*r*r;
}
```

Here we have a file called “main.c”, which contains my main function (the file containing the main function is usually named “main” by convention). I have another c source file called “circle.c”, which contains a function for computing the area of a circle. I also have a shared header file, “circle.h”, which contains the function prototype for the area function and a definition for the value of PI. When compiling, each “.c” file gets compiled separately, so the preprocessor will be ran for both “main.c” and “circle.c”. Let’s see what these files look like after the preprocessor has run:

main.c

2. C Programming

```
float circleArea(float r);
int main(){
    float r = 3.5;
    float area = circleArea(r);
    return 0;
}
```

circle.c

```
float circleArea(float r);
float circleArea(float r){
    return 3.14*r*r;
}
```

What we observe here is that the function prototype for the area has been appended to the top of each file, and anywhere the label “PI” was found, it was replaced by its defined value, 3.14. In this example, that was only a single location in the circleArea() function. Now that all the preprocessor macros have been resolved, the code is ready to be sent to the compiler.

2.13.2. The Compiler

The compiler is responsible for interpreting the C source code into its assembly language equivalent. We haven’t gone into the details of assembly, and we won’t in this book, but let’s take a look at what the compiler will do to each of our files. Note that the “.c” extension will be replaced with “.s” for each respective assembly file, as “.s” is the typical assembly file extension:

main.s

```

main:
    push    {r7, lr}
    sub     sp, sp, #8
    add     r7, sp, #0
    ldr     r3, .L7
    str     r3, [r7, #4]
    ldr     r3, [r7, #4]
    adds    r0, r3, #0
    bl      circleArea
    adds    r3, r0, #0
    str     r3, [r7]
    movs    r3, #0
    movs    r0, r3
    mov     sp, r7
    add     sp, sp, #8
    pop     {r7, pc}

```

circle.s

```

circleArea:
    push    {r7, lr}
    sub     sp, sp, #8
    add     r7, sp, #0
    str     r0, [r7, #4]
    ldr     r1, .L3
    ldr     r0, [r7, #4]
    bl      __aeabi_fmul
    adds    r3, r0, #0
    ldr     r1, [r7, #4]
    adds    r0, r3, #0
    bl      __aeabi_fmul
    adds    r3, r0, #0
    adds    r0, r3, #0

```

2. C Programming

```
mov    sp, r7
add    sp, sp, #8
pop    {r7, pc}
```

Despite `circleArea()` being a small function in C, the resulting assembly is not trivial. This is due to the fact that it uses floating-point operations. The Cortex-M0 core in the STM32F091RC does not include a hardware floating-point unit, so the compiler must instead call a software floating-point routine, `__aeabi_fmul`, to perform each multiplication. Therefore, programmers prefer integer operations, when possible, as they are considerably smaller and easier for the processor to handle from a performance perspective.

2.13.3. The Assembler

The assembler takes the assembly files created by the compiler and converts them into object files. Object files are part machine code, part symbols. The symbols represent things that aren't available in the source and must be resolved within other files, such as the implementation of the “`circleArea()`” function call in `main`. `Main` understands that “`circleArea()`” is a function with certain parameters because of the prototype that was included in the “`circle.h`” header file. `Main` will need to generate a symbol for that function so that it can be resolved in the next step: the linker. The location where the symbol will belong can be seen in the assembly code above for `main`; the line containing “`bl circleArea`” will have a symbol to be linked for “`circleArea`”.

2.13.4. The Linker

The linker's job is to take all the object files, resolve the symbols, and create the executable. This is the stage where all unimplemented functions

across all the object files will be tied to their appropriate definitions. Once the linker is completed, the executable file is produced, containing the machine code to be ran by the processor.

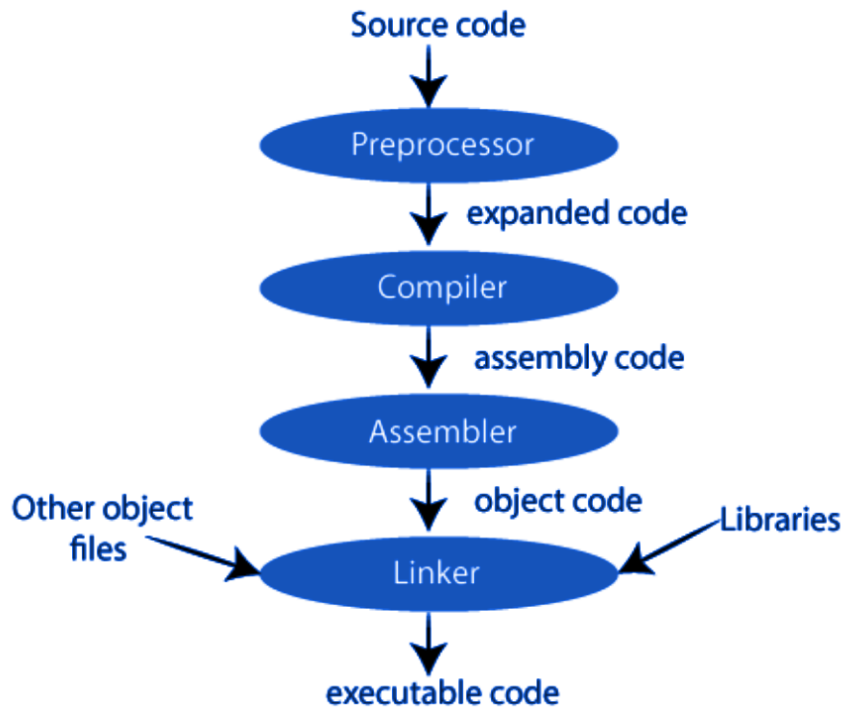


Figure 2.10.: The compilation process

2.13.5. The Loader

A fourth step exists for embedded system applications, which is the loader. The loader is a tool responsible for placing the executable into the microcontroller's memory. Small-scale microcontrollers often simply store their

2. C Programming

program in the text region of memory, and this region must be completely re-written when updating the code.

2.13.6. Cross Compilers

We've explored the process of compilation, but there is one more important detail to acknowledge when it comes to building C programs for embedded systems, and that is the processor that is building the program and what processor the program is to be ran on. In typical software development, when writing applications to be ran on a desktop or laptop computer, the type of machine the code is being built on is also the same machine the code will be intended to be ran. Therefore, most compilers are made to build for the machine doing the building, but that is not the case for embedded systems.

When writing programs for microcontrollers, such as the STM32F091RC, we are writing and building the code on a desktop or laptop, which is most likely an x86 architecture, and running the code on an Arm architecture. Therefore, we need a special compiler known as a cross-compiler. A cross compiler will build code on one machine to be ran on a machine of a different architecture. The machine doing the building is known as the **host** machine, and the machine the program is built for is known as the **target** machine.

2.14. Integrated Development Environments

To aid with the complexity of running cross compilers, editing code, and the many other needs required for creating programs for embedded systems, programmers often use an **integrated development environment** (IDE). IDEs are applications which usually include a variety of tools to help with software development. For embedded systems, this typically includes the following:

2.14. Integrated Development Environments

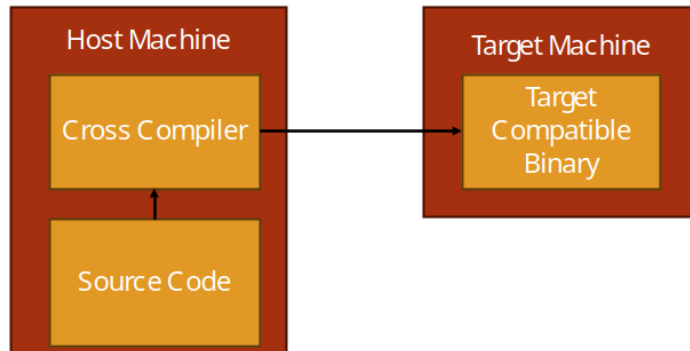


Figure 2.11.: The Cross-Compilation Process

- Text Editor and Syntax Highlighter
- Build Tools
- Debugging Tools

Even more features may come in other IDEs, but for embedded systems these are the most common.

2.14.1. STM32CubeIDE

When developing for the STM32F091RC, or any STM32 microcontroller, STM32CubeIDE is the free IDE provided by STMicroelectronics to support their products. STM32CubeIDE is built on the Eclipse C/C++ Development Tooling (CDT) platform and contains all the tools listed in the previous section, plus a few more, such as STM32CubeMX-based project and peripheral configuration, which can generate initialization code for a project based on the microcontroller and peripherals selected.

2. C Programming

2.14.1.1. STM32CubeIDE UI Overview

Let's take a look at the user interface (UI) of the STM32CubeIDE environment. We'll start by highlighting the large section in the center of the window, the text editor. The locations of these windows shown here are the defaults, but they can be moved around per the user's preferences. The text editor provides syntax highlighting and a space to edit the code. It also can support the addition of breakpoints, which when in debugging mode will allow the processor to stop execution at the line of code marked with a breakpoint and allow the user to inspect things such as memory content and variable values.

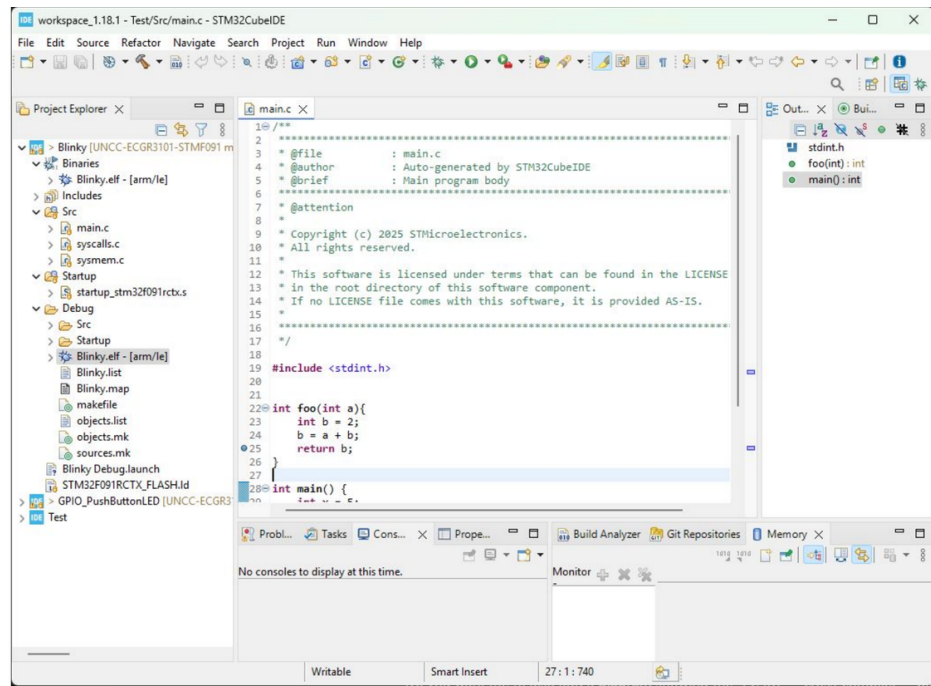


Figure 2.12.: The STM32CubeIDE, showing the project explorer, source editor, and outline panels.

2.14. Integrated Development Environments

On the left side you can see several directories with different items inside. This is the **Project Explorer** and it displays the contents of the workspace directory on your computer. The workspace directory is a location you specify where STM32CubeIDE will store your projects. A project is a collection of source files (.c, .h), startup code, linker scripts, and build configuration that tell the Arm cross compiler what microcontroller the project is building for and how to build it. By organizing everything into a project directory, a lot of the intricacies and complications of cross compilation is managed by STM32CubeIDE rather than the user having to remember all the details.

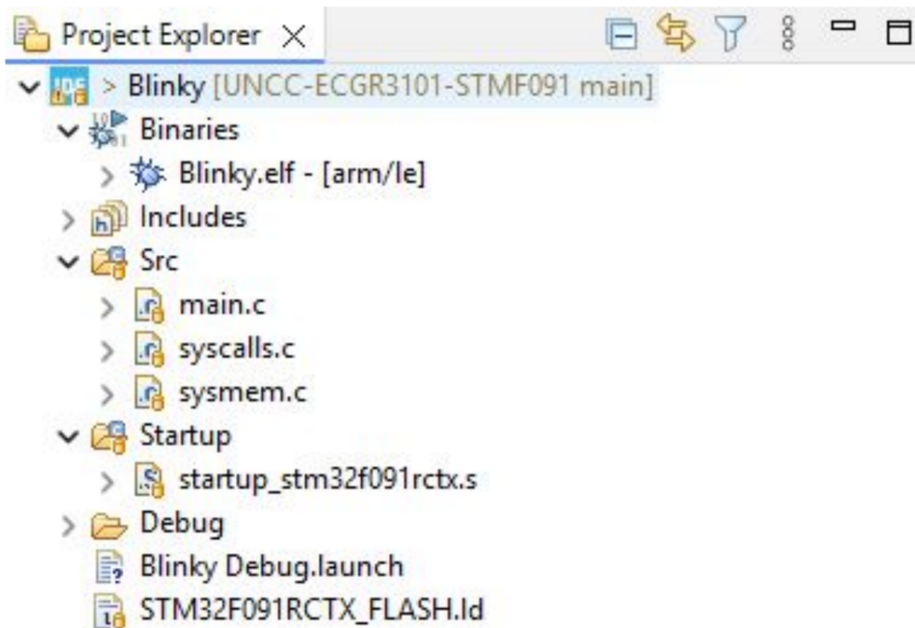


Figure 2.13.: The Project Explorer for a “Blinky” project, showing the Drivers (CMSIS and HAL libraries), Startup code, and Src/Inc application source folders that STM32CubeIDE generates for a new project.

2. C Programming

A newly generated project's Project Explorer will typically contain the following folders:

- **Drivers** – contains the CMSIS (Cortex Microcontroller Software Interface Standard) headers and the STM32 Hardware Abstraction Layer (HAL) library, which provides functions for configuring and using the microcontroller's peripherals.
- **Startup** – contains the startup assembly file, responsible for initializing the stack pointer and calling `main` after a reset.
- **Src** and **Inc** – contain the project's own application source (.c) and header (.h) files, including `main.c`.

On the top, we have the toolbar menu, which gives the user access to all of STM32CubeIDE's features and tools. The toolbar has everything organized under the labels: file, edit, source, refactor, navigate, search, project, run, window, and help. There are a lot of features packed under these labels, but for convenience, several of the more common ones are broken out underneath with several icons to indicate their purpose. We'll highlight the most basic ones here:

- Hammer Icon – builds the active project
- Bug Icon – builds the active project, uploads it to the target, and begins a debugging session
- Play Button – continues the current debug session
- Pause Button – pauses the current debug session
- Stop Button – stops the current debug session
- Step Into – steps into the next function or line of code in the debug session
- Step Over – steps over the next instruction in the debug session

2.15. Recap

New projects for the STM32F091RC are typically started using STM32CubeMX (either as a standalone tool, or integrated directly within STM32CubeIDE). STM32CubeMX allows the user to select the target microcontroller (or development board, such as the Nucleo-F091RC), configure its clocks and peripherals through a graphical interface, and then generate the initialization code and project structure automatically – rather than needing to hand-write the low-level register configuration for each peripheral, as would be required with just a bare compiler.

2.15. Recap

In this chapter we have learned why we program embedded systems in C, the basic syntax of C, and how C variables related to memory in terms of bytes occupied and location. We also have been introduced to the generic computer memory map and what each region is used for and how it operates.

Part II.

**Digital I/O and Program
Structure**

3. General Purpose Input/Output (GPIO)

3.1. Learning Objectives

In this chapter we will learn:

- What peripherals are
- How to find and use microcontroller documentation
- What General Purpose Input/Output is
- How to control peripherals by writing to memory-mapped registers
- The STM32 GPIO register set
- Bit masking

3.2. Interfacing with the Outside World

A microcontroller isn't very useful if it can't interface with the world around it. In order to read a button press, illuminate an LED, or communicate with another chip, the microcontroller needs specialized hardware built into it beyond the processor and memory we've discussed so far. This specialized hardware is referred to as a **peripheral**. A peripheral is a piece of hardware, separate from the processor core, that is included in the

3. General Purpose Input/Output (GPIO)

microcontroller to provide some additional capability, typically related to input or output.

The STM32F091RC, like most microcontrollers, includes many different peripherals, each specialized for a different task. Some peripherals produce or read digital voltages, some convert analog voltages to digital values (and vice versa), some keep track of time, and others communicate with external devices using a specific protocol. Figure 3.1 shows an illustration of some of the peripherals commonly found on a microcontroller.

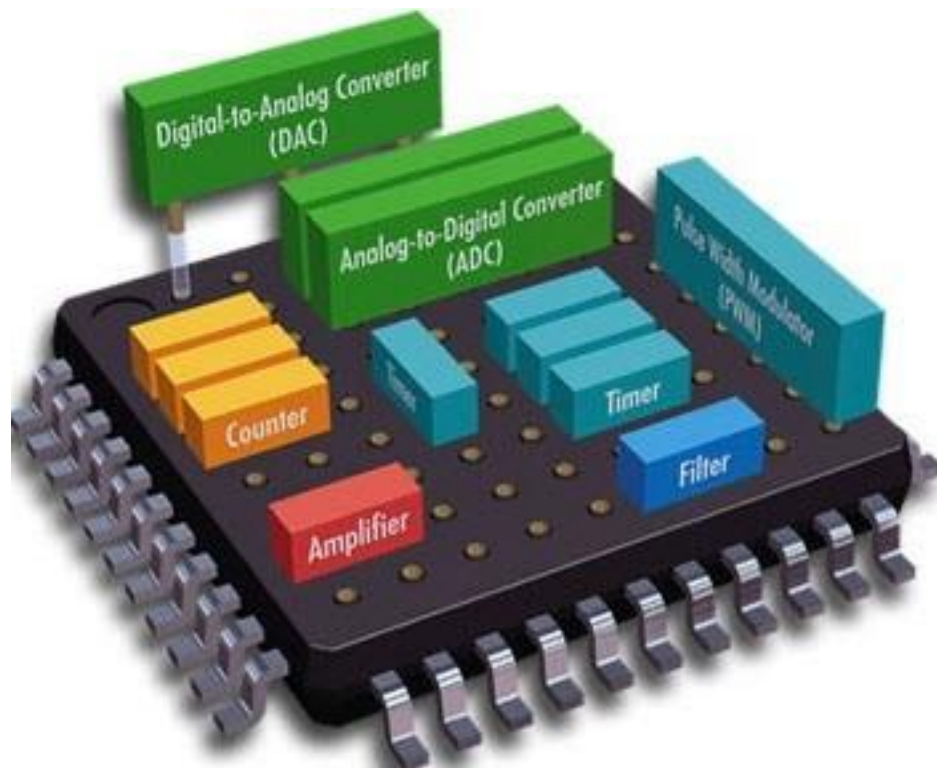


Figure 3.1.: Peripherals are specialized hardware blocks built into the microcontroller alongside the processor core

3.3. Microcontroller Documentation

In this chapter, we will focus on the most fundamental of these peripherals: the **General-Purpose Input/Output** (GPIO) peripheral. Before we get into the specifics of GPIO, however, let's discuss where we will find the information necessary to use any peripheral on the STM32F091RC.

3.3. Microcontroller Documentation

Nearly everything we do as embedded systems programmers requires us to consult documentation. Since a peripheral's behavior is dictated by hardware, and every microcontroller's hardware is designed a little differently, we cannot simply guess how to configure it. Instead, we must rely on documentation provided by the manufacturer, in our case, STMicroelectronics.

It's tempting to think of this documentation as a single document, but in practice, you'll find that manufacturers split their documentation into several different documents, each covering a different level of detail. For the STM32F091RC on our NUCLEO-F091RC development board, we will regularly reference three different kinds of documents:

- Documentation specific to our development board (the NUCLEO-F091RC)
- Documentation specific to our microcontroller family (the STM32F0 series)
- Documentation specific to our exact microcontroller (the STM32F091RC)

Each of these documents serves a different purpose, and knowing which one to open for a given question will save you a lot of time.

3. General Purpose Input/Output (GPIO)

3.3.1. Development Board Documentation

The development board documentation describes the board itself, not the microcontroller. This document will help us identify the specifics of our particular development board, such as where the buttons and LEDs are connected, how the on-board debugger works, and the layout of the board's schematic. For our NUCLEO-F091RC, this information is found in ST's *UM1724* user manual, shown in Figure 3.2.

3.3.2. Microcontroller Family Documentation

The microcontroller family documentation, more commonly called the **reference manual**, contains details that are common across an entire family of microcontrollers. This is usually the document we will reference the most while programming, as it contains detailed descriptions of each peripheral and its associated registers. For the STM32F091RC, this is ST's *RM0091* reference manual, shown in Figure 3.3, which covers the entire STM32F0x1/F0x2/F0x8 family of microcontrollers, of which the STM32F091RC is a member.

3.3.3. Microcontroller-Specific Documentation

Finally, the **datasheet** is the documentation most specific to our exact chip. This document typically contains mechanical drawings of the physical package, the pinout diagram, and electrical characteristics, such as the operating voltage range and current limits. For the STM32F091RC, this information can be found in the datasheet shown in Figure 3.4.

Note

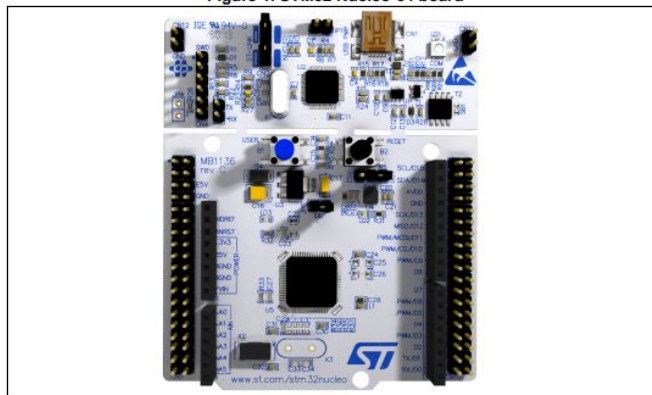
Throughout this chapter, we will regularly point out which of these three documents contains the information we're discussing, so you

3.3. Microcontroller Documentation

Introduction

The STM32 Nucleo-64 boards based on the MB1136 reference board (NUCLEO-F030R8, NUCLEO-F070RB, NUCLEO-F072RB, NUCLEO-F091RC, NUCLEO-F103RB, NUCLEO-F302R8, NUCLEO-F303RE, NUCLEO-F334R8, NUCLEO-F401RE, NUCLEO-F410RB, NUCLEO-F411RE, NUCLEO-F446RE, NUCLEO-L010RB, NUCLEO-L053R8, NUCLEO-L073RZ, NUCLEO-L152RE, NUCLEO-L452RE, NUCLEO-L476RG) provide an affordable and flexible way for users to try out new concepts and build prototypes with the STM32 microcontrollers in the LQFP64 package, choosing from the various combinations of performance, power consumption, and features. The ARDUINO® Uno V3 connectivity support and the ST morpho headers provide an easy means of expanding the functionality of the Nucleo open development platform with a wide choice of specialized shields. The STM32 Nucleo boards do not require any separate probe as they integrate the ST-LINK/V2-1 debugger and programmer. The STM32 Nucleo boards come with the comprehensive free software libraries and examples available with the STM32Cube MCU Packages.

Figure 1. STM32 Nucleo-64 board



Picture is not contractual.

Figure 3.2.: The UM1724 user manual documents the NUCLEO-64 family of development boards, including the NUCLEO-F091RC

3. General Purpose Input/Output (GPIO)

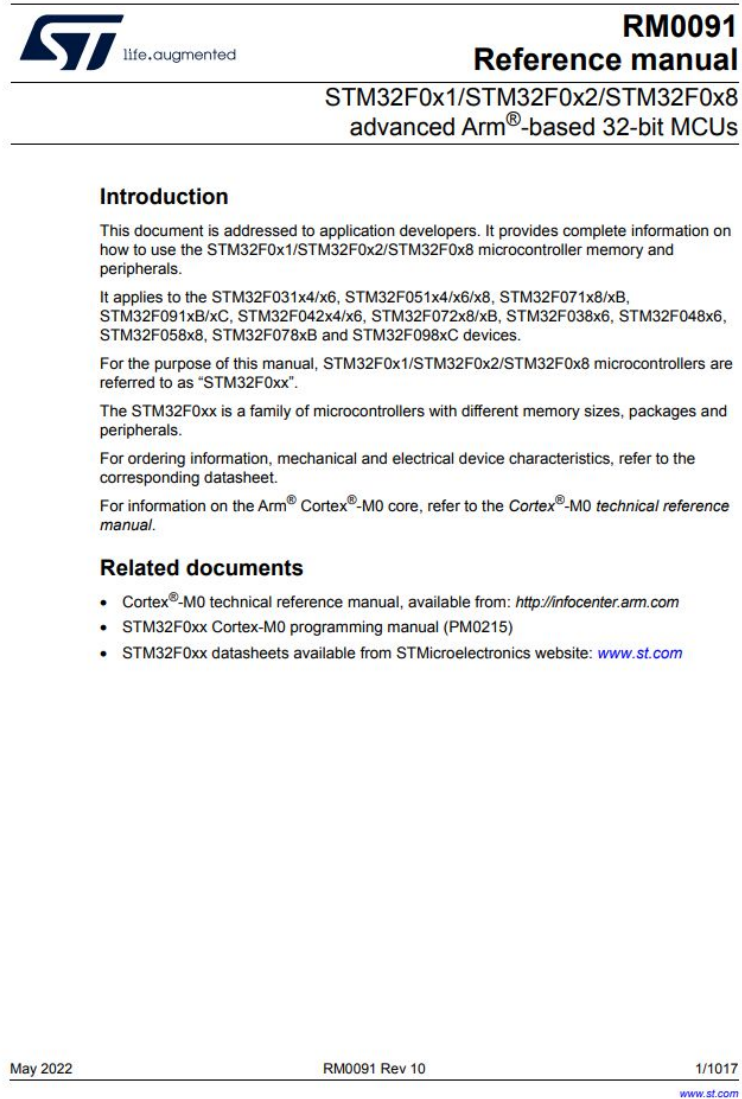


Figure 3.3.: The RM0091 reference manual documents the peripherals shared across the entire STM32F0 family

3.3. Microcontroller Documentation



STM32F091xB STM32F091xC

ARM®-based 32-bit MCU, up to 256 KB Flash, CAN, 12 timers, ADC, DAC, and comm. interfaces, 2.0 - 3.6V

Datasheet - production data

Features

- Core: ARM® 32-bit Cortex®-M0 CPU, frequency up to 48 MHz
- Memories
 - 128 to 256 Kbytes of Flash memory
 - 32 Kbytes of SRAM with HW parity
- CRC calculation unit
- Reset and power management
 - Digital & I/Os supply: $V_{DD} = 2.0\text{ V}$ to 3.6 V
 - Analog supply: $V_{DDA} = V_{DD}$ to 3.6 V
 - Power-on/Power down reset (POR/PDR)
 - Programmable voltage detector (PVD)
 - Low power modes: Sleep, Stop, Standby
 - V_{BAT} supply for RTC and backup registers
- Clock management
 - 4 to 32 MHz crystal oscillator
 - 32 kHz oscillator for RTC with calibration
 - Internal 8 MHz RC with x6 PLL option
 - Internal 40 kHz RC oscillator
 - Internal 48 MHz oscillator with automatic trimming based on ext. synchronization
- Up to 88 fast I/Os
 - All mappable on external interrupt vectors
 - Up to 69 I/Os with 5V-tolerant capability and 19 with independent supply V_{DDIO2}
- 12-channel DMA controller
- One 12-bit, $1.0\text{ }\mu\text{s}$ ADC (up to 16 channels)
 - Conversion range: 0 to 3.6 V
 - Separate analog supply: 2.4 V to 3.6 V
- One 12-bit D/A converter (with 2 channels)
- Two fast low-power analog comparators with programmable input and output
- Up to 24 capacitive sensing channels for touchkey, linear and rotary touch sensors
- Calendar RTC with alarm and periodic wakeup from Stop/Standby

 LQFP100 14x14 mm
 LQFP64 10x10 mm
 LQFP48 7x7 mm
 UQFPN48 7x7 mm
 UFBGA100 7x7 mm
 UFBGA64 5x5 mm
 WLCSP64 3.3x3.6mm

- 12 timers
 - One 16-bit advanced-control timer for 6 channel PWM output
 - One 32-bit and seven 16-bit timers, with up to 4 IC/OC, OCN, usable for IR control decoding or DAC control
 - Independent and system watchdog timers
 - SysTick timer
- Communication interfaces
 - Two I²C interfaces supporting Fast Mode Plus (1 Mbit/s) with 20 mA current sink, one supporting SMBus/PMBus and wakeup
 - Up to eight USARTs supporting master synchronous SPI and modem control, three with ISO7816 interface, LIN, IrDA, auto baud rate detection and wakeup feature
 - Two SPIs (18 Mbit/s) with 4 to 16 programmable bit frames, and with I²S interface multiplexed
 - CAN interface
- HDMI CEC wakeup on header reception
- Serial wire debug (SWD)
- 96-bit unique ID
- All packages ECOPACK®2

Table 1. Device summary

Reference	Part number
STM32F091xB	STM32F091CB, STM32F091RB, STM32F091VB
STM32F091xC	STM32F091CC, STM32F091RC, STM32F091VC

January 2017
DocID026284 Rev 4
1/128

This is information on a product in full production.
www.st.com

Figure 3.4.: The datasheet contains details specific to the STM32F091xB/xC devices, such as pinout and electrical characteristics

3. General Purpose Input/Output (GPIO)

can get comfortable navigating between them.

3.4. General Purpose Input Output

Now that we know where to look for information, let's discuss the GPIO peripheral itself. The GPIO peripheral allows the microcontroller to read or set digital voltages on the pins of the chip. This gives us the ability to create some of the most fundamental forms of input and output, such as illuminating an LED or reading a button press.

Let's recall what is meant by a “digital” voltage. A digital voltage represents either a 1 or a 0. A 0 is typically represented by 0 volts, and a 1 is represented by the microcontroller's operating voltage. When speaking generally about these voltages, we use the terms **high** and **low** to refer to whatever voltage represents a 1 and 0 in the system, respectively. Voltages between these two extremes may be interpreted inconsistently by the microcontroller and should be avoided when working with GPIO.

3.4.1. How to Find Operating Voltage

The operating voltage of a microcontroller is defined in its datasheet. The STM32F091RC typically operates within a range of 2.0 V to 3.6 V. The NUCLEO-F091RC development board is powered over USB, which supplies 5 V. This 5 V is regulated down to 3.3 V before being supplied to the microcontroller, so 3.3 V is the voltage we should expect to see representing a digital high on this board.

3.4.2. Controlling an LED

Let's consider an example of illuminating an LED using a GPIO pin. Say we have a GPIO peripheral controlling a pin labeled “P1,” which is

3.4. General Purpose Input Output

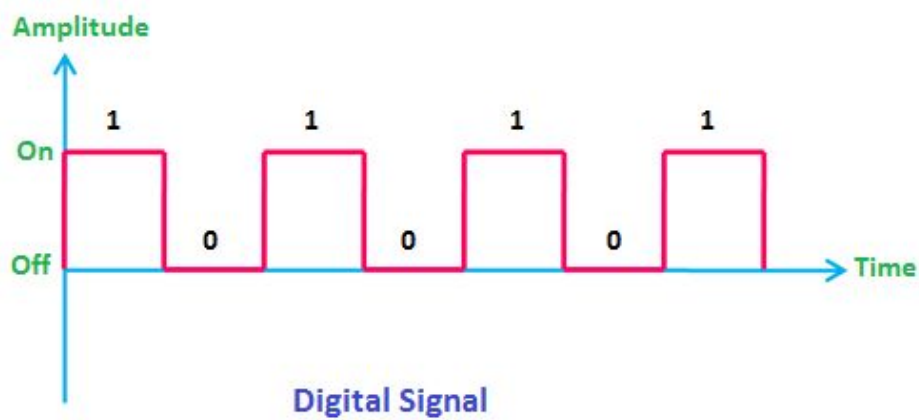


Figure 3.5.: A digital signal alternates between a high (on) and low (off) voltage

Figure 10. Power supply input from 5V_EXT (5 V, 1.3 A maximum)

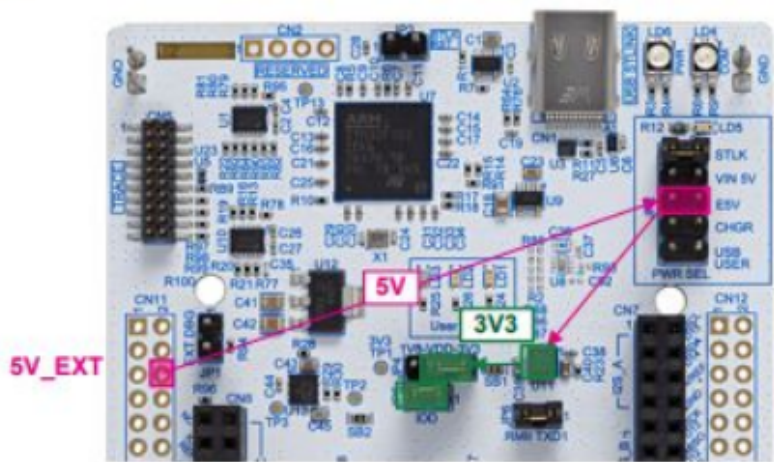


Figure 3.6.: Power Regulation for the NUCLEO-F091RC

3. General Purpose Input/Output (GPIO)

connected to a small current-limiting resistor in series with an LED, which is then connected to ground, as shown in Figure 3.7.

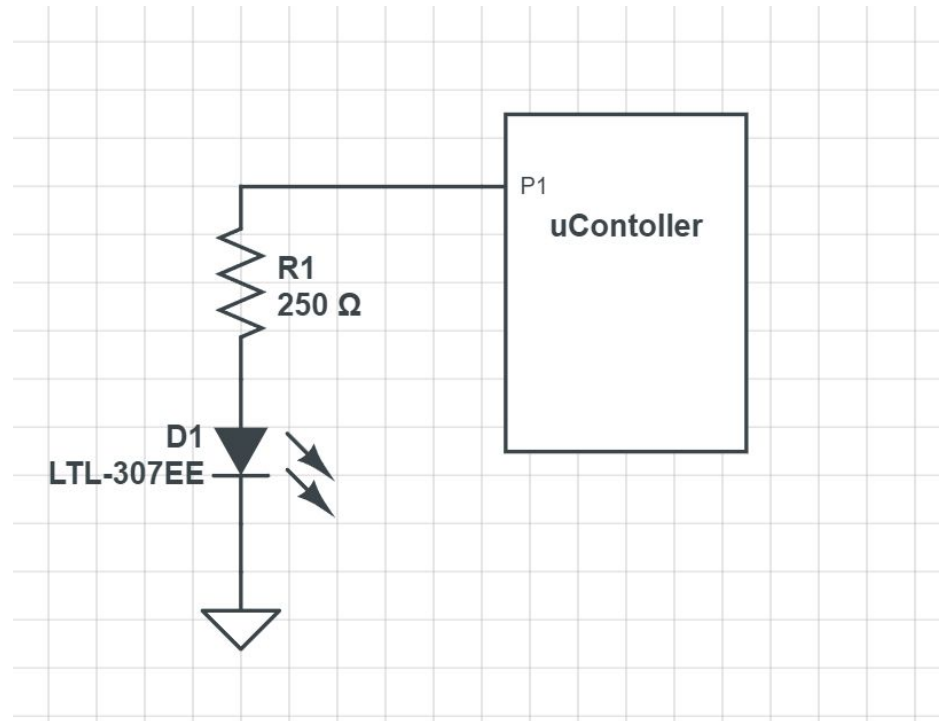


Figure 3.7.: A GPIO pin driving an LED through a current-limiting resistor

When the GPIO provides a high voltage on P1, current flows from P1, through the resistor, through the LED, and to ground, causing the LED to illuminate. When the GPIO writes a low voltage (0 V), there is no longer any electrical potential across the LED, since ground is also at 0 V. With no potential difference, no current flows, and the LED turns off.

3.4.3. Reading a Button Press

Now let's consider reading a button press. To do this, we need a switch that can control whether a high or low voltage is presented to a GPIO input pin. We'll connect P1 to a resistor that is wired to a high voltage source, and also to a switch that connects the pin to ground, as shown in Figure 3.8.

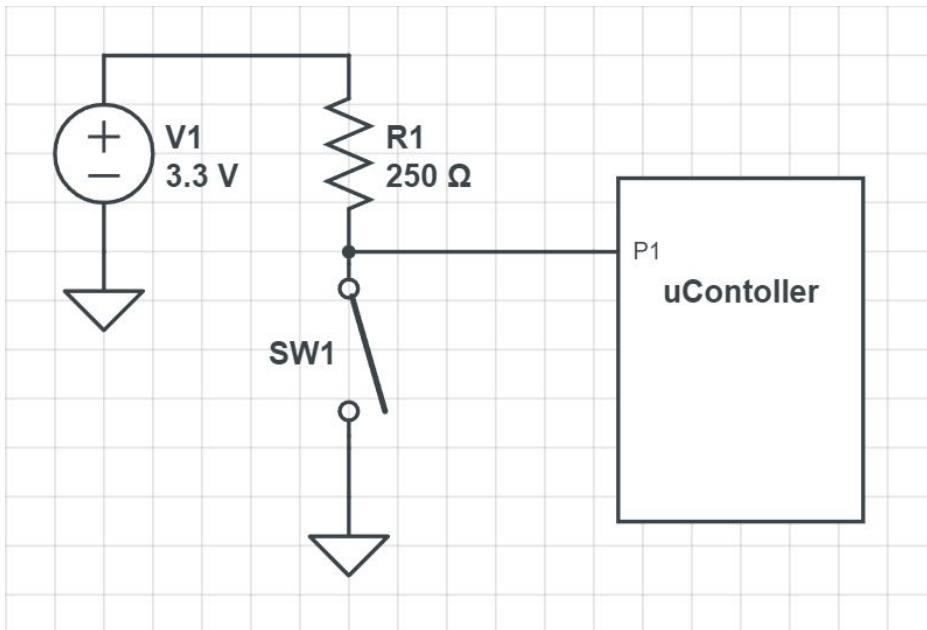


Figure 3.8.: A GPIO pin reading the state of a switch through a pull-up resistor

When the switch is closed, P1 is directly connected to ground and reads 0 V; nearly all the voltage is dissipated across the resistor, and negligible current flows into the microcontroller. When the switch is open, there is no longer a direct path to ground, so current flows into P1 instead. Since the input of a GPIO pin has extremely high resistance, nearly all the voltage is

3. General Purpose Input/Output (GPIO)

dissipated across the microcontroller's input rather than the resistor, and P1 reads a high voltage.

We've now covered the two most fundamental input/output scenarios for an embedded system. Let's see how this is achieved on the STM32F091RC.

3.5. Peripherals and the Memory Map

A processor is really only capable of reading and writing data to its memory. Generally speaking, programs are sequences of instructions that moves data out, performs operations on that data, then writes it back.

In order for our program instructions to control a peripheral, that peripheral must therefore appear as a memory location so our instructions can write to it. Each peripheral contains many **control registers**, which behave like any other memory location, except that the values written to them dictate how the peripheral's hardware behaves. For GPIO, this might mean configuring whether a pin is an input or an output, writing a high or low voltage to a pin, or reading whether a pin currently holds a high or low voltage.

Since the only thing an instruction can interact with is memory, peripherals need to appear as memory! Specific memory addresses are wired directly to the peripheral hardware, so writing to those addresses controls entirely separate pieces of hardware, completely independent from RAM.

We can access these memory-mapped locations using a pointer. If we have a pointer that points to the memory address associated with a peripheral's register, we can dereference it to write data directly to that peripheral and control it.

```
uint16_t value = 0x0010;
uint16_t* p = (uint16_t*) 0x48000014;
*p = value;
```

3.5. Peripherals and the Memory Map

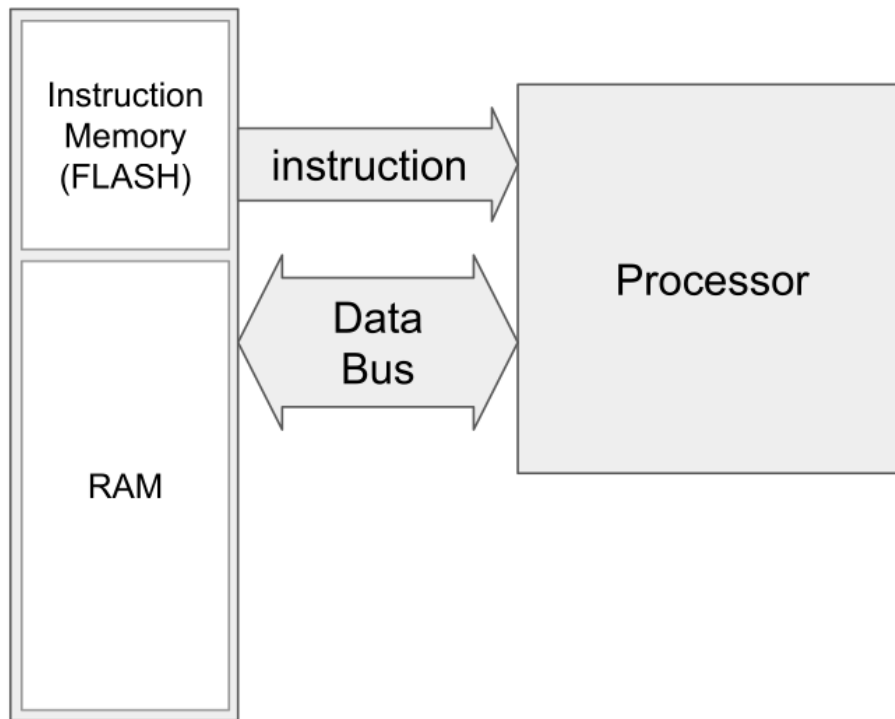


Figure 3.9.: The processory reads instructions from memory, which then read and write to the data in RAM.

3. General Purpose Input/Output (GPIO)

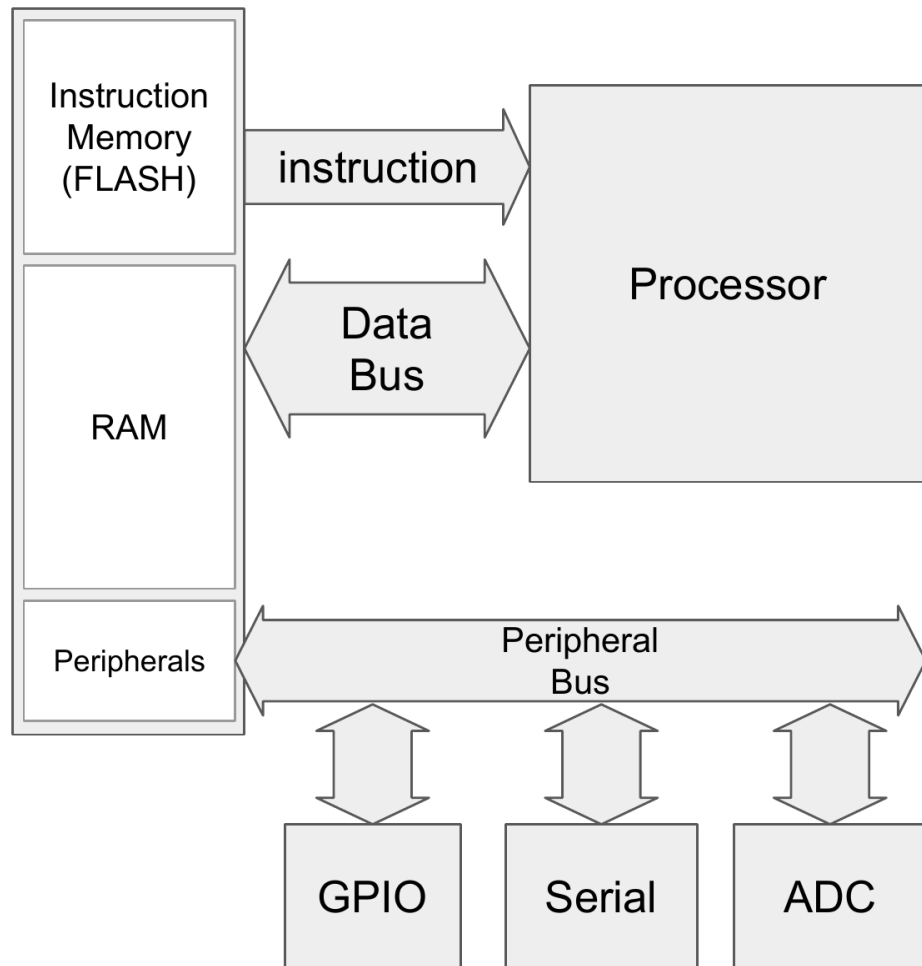


Figure 3.10.: Peripherals are wired into the same address space that instructions use to access memory, allowing a single memory bus to reach both RAM and hardware peripherals

3.5.1. The STM32F091 Memory Map

Each microcontroller has its own memory map, so to know where a peripheral's control registers reside, we consult the reference manual. Figure 3.11 shows the memory map for the STM32F091RC. The regions we're already familiar with, such as the stack, heap, and program instructions, are all found within the "CODE" and "SRAM" regions here. New regions, labeled APB, AHB1, and AHB2, are associated with the various peripherals, including GPIO.

3.6. GPIO Organization

On the STM32, a single GPIO peripheral can only control up to 16 pins. Since the STM32F091RC has far more pins than this, the microcontroller includes several copies of the GPIO peripheral, each controlling a different set of pins. Each copy of the GPIO peripheral is referred to as a **Port**, and each port is identified by a letter (A, B, C, and so on).

Individual pins on a port are identified by number, from 0 to 15. To refer to a specific pin, we use the notation **PXN**, where X is the port letter and N is the pin number. For example, "PA5" refers to Port A, Pin 5, and "PC13" refers to Port C, Pin 13.

3.6.1. Which Pins are GPIO?

Not every pin on the microcontroller can be controlled by the GPIO peripheral; some pins are dedicated to power, ground, or the reset line. To determine which pins are available for GPIO, we consult the datasheet. The datasheet includes a pinout diagram of the chip's package, such as the one shown in Figure 3.13. Pins labeled with the PXN format, such as PA5 or PC13, are the ones controlled by GPIO. Many of these pins can also be

3. General Purpose Input/Output (GPIO)

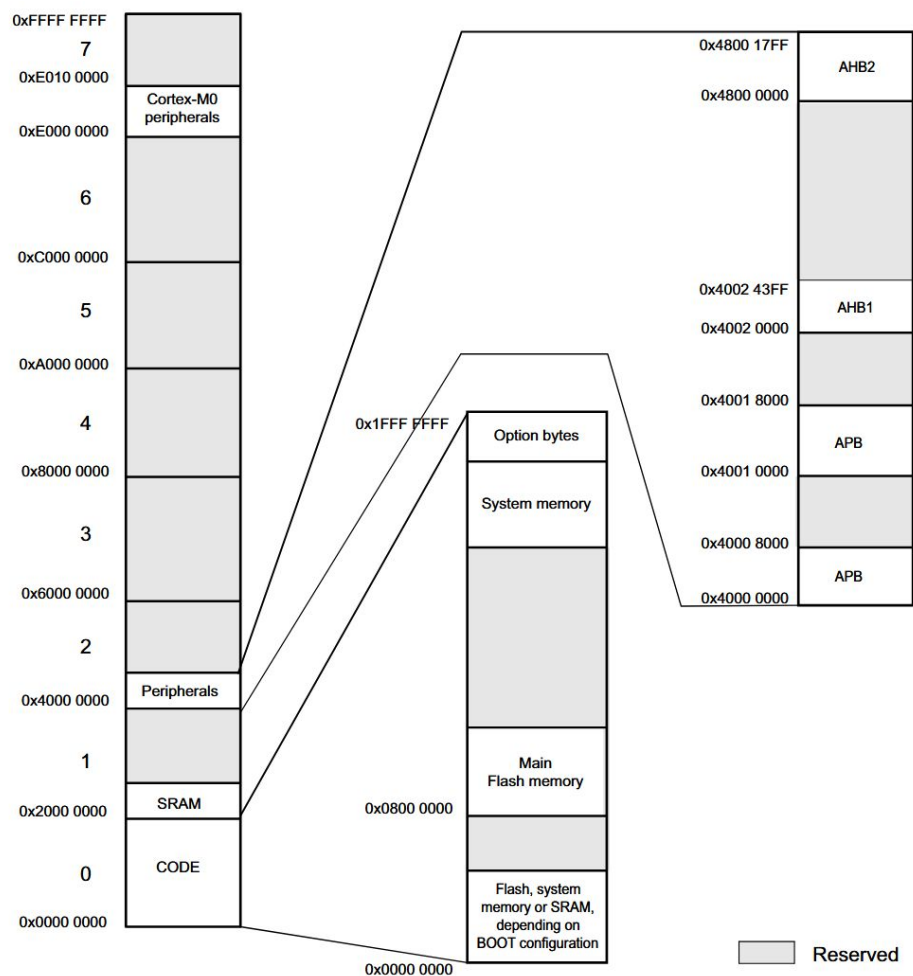


Figure 3.11.: The STM32F091RC memory map, showing the regions dedicated to CODE, SRAM, and the memory-mapped peripheral buses

3.7. Controlling the GPIO

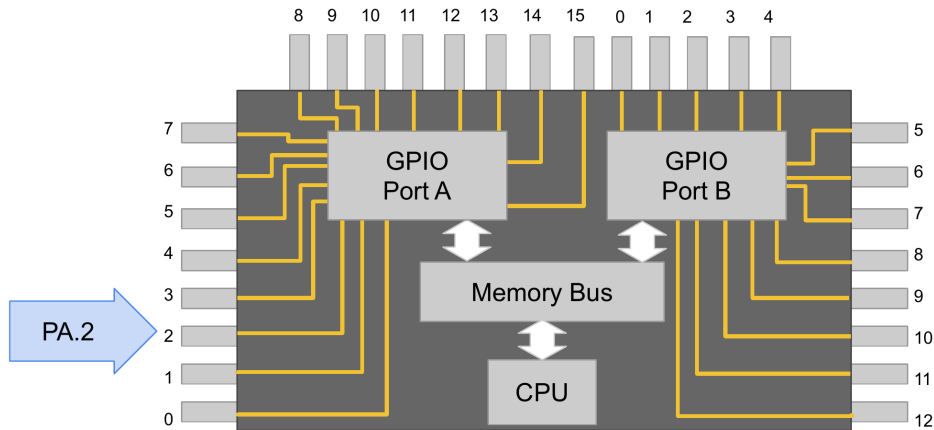


Figure 3.12.: Each GPIO port is a separate copy of the GPIO peripheral hardware, controlling up to 16 pins of the chip, and each is connected to the CPU through the same memory bus

shared with other peripherals, which we'll discuss further when we cover pin alternate functions in a later chapter.

3.6.2. Dev Board Pinout

Since we're using a development board, the pins of the microcontroller's package have already been broken out to header pins along the edges of the board. The NUCLEO-F091RC's user manual includes diagrams, shown in Figure 3.14 and Figure 3.15, that make it easy to identify which port and pin is connected to each header.

3.7. Controlling the GPIO

Now that we know the why and where of GPIO, let's discuss the how. Most microcontrollers' GPIO peripherals have four core registers used to

3. General Purpose Input/Output (GPIO)

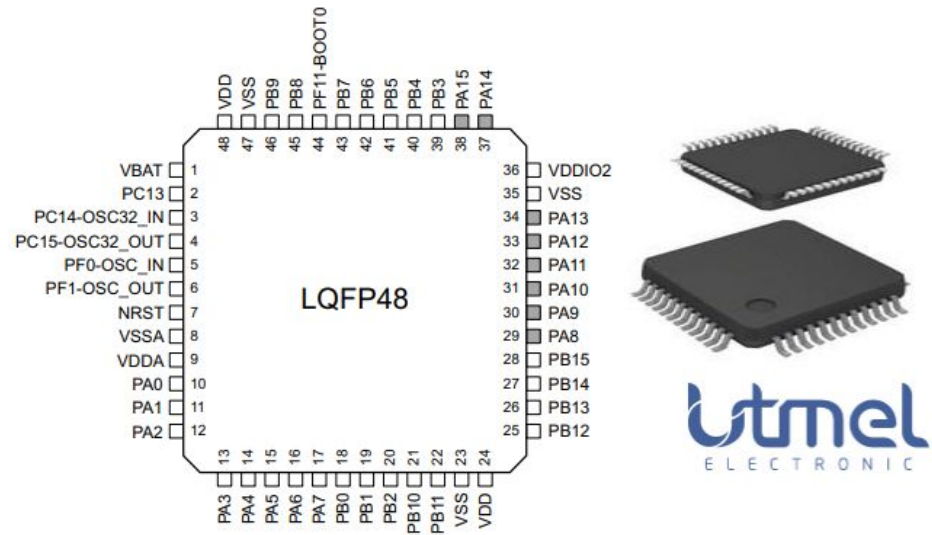


Figure 3.13.: An example pinout diagram from the datasheet. Pins labeled in the PXN format are controlled by the GPIO peripheral

3.7. Controlling the GPIO

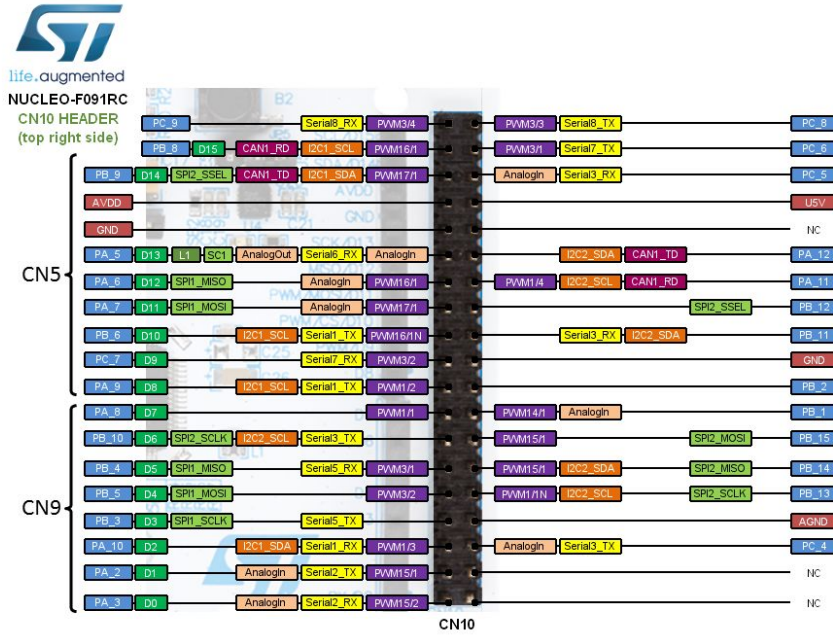


Figure 3.14.: NUCLEO-F091RC header pinout (CN7/CN8, left side of the board)

3. General Purpose Input/Output (GPIO)

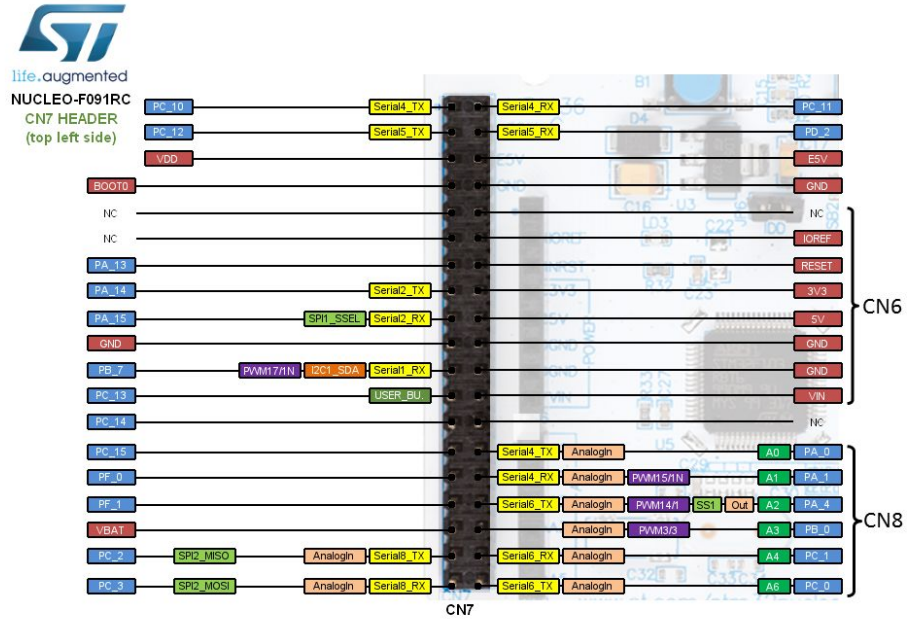


Figure 3.15.: NUCLEO-F091RC header pinout (CN5/CN9/CN10, right side of the board)

3.8. Example Application: Blink an LED

control them:

Directional Register (MODER) Configures a pin as an input or output

Output Register (ODR) Sets the value of a pin configured as an output
(high or low)

Input Register (IDR) Reads the current value of a pin

Resistor Enable Register (PUPDR) Enables a pull-up or pull-down resistor
on an input pin

Let's walk through a complete example using each of these registers:
blinking the green LED (LD2) built into the NUCLEO-F091RC.

3.8. Example Application: Blink an LED

3.8.1. Driving an LED with GPIO

The development board has an LED already connected to a GPIO pin. To find out which pin, we consult the development board's schematic in the UM1724 user manual, shown in Figure 3.16. From this, we can see that a green LED is connected to PA5.

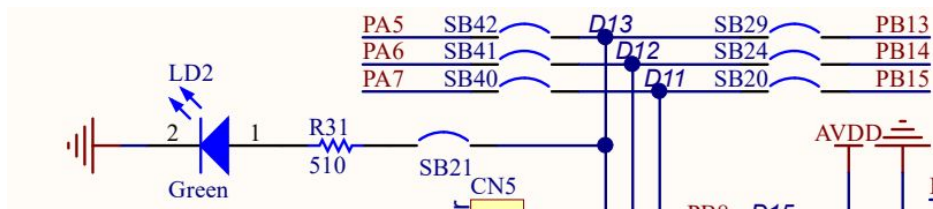


Figure 3.16.: The green user LED (LD2) on the NUCLEO-F091RC is connected to PA5

3. General Purpose Input/Output (GPIO)

3.8.2. Enabling the Peripheral Clock

Before we can configure any of GPIOA's registers, there's an important STM32-specific step we must take: enabling the peripheral's clock. To conserve power, most peripherals on the STM32 are disabled by default and are not supplied with a clock signal until we explicitly enable them. This is controlled through the **Reset and Clock Control** (RCC) peripheral. Each GPIO port has a corresponding enable bit in the `RCC_AHBENR` register. GPIOA's clock enable bit is bit 17 (`IOPAEN`). If we forget this step, writes to GPIOA's registers will have no effect, since the peripheral isn't receiving a clock signal at all!

```
volatile unsigned int* rcc_ahbenr = (unsigned int*) 0x40021014;  
*rcc_ahbenr = 0x00020000; // Enable GPIOA (IOPAEN, bit 17)
```

3.8.3. Setting the Mode

In order to set the voltage on a GPIO pin high, we must first configure that pin as an output. This is done using the mode register, `GPIOx_MODER`. From the reference manual, we can see that the mode register is 32 bits wide, and unlike a single enable bit, a *pair* of bits is used to configure the mode of each pin, as shown in Figure 3.17.

Each pin's two bits can be set to one of four values:

- 00: Input mode (the reset/default state)
- 01: General purpose output mode
- 10: Alternate function mode
- 11: Analog mode

For example, to set PA5 as an output, we need to write 01 to bits 11 and 10 (pin 5's field within the register):

3.8. Example Application: Blink an LED

8.4.1 GPIO port mode register (GPIOx_MODER) (x =A to F)

Address offset:0x00

Reset value: 0x2800 0000 for port A

Reset value: 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figure 3.17.: The GPIO port mode register (GPIOx_MODER) uses two bits per pin to select between input, output, alternate function, and analog modes

3. General Purpose Input/Output (GPIO)

```
volatile unsigned int* gpioa_moder = (unsigned int*) 0x48000000;  
*gpioa_moder = 0x00000400; // MODER5 = 01 -> bits [11:10]
```

i Note

Sometimes a pin is shared with another peripheral, such as a timer or an ADC. The GPIO peripheral acts as the “gatekeeper” for the pin and must be configured with the alternate function mode (10) to let those other peripherals’ signals pass through to the pin.

3.8.4. Output Register

The output register, `GPIOx_ODR`, sets the value of each pin that has been configured as an output. Unlike the mode register, each bit in the output register corresponds to a single pin, as shown in Figure 3.18. Writing a 1 sets the pin high, and writing a 0 sets it low.

To write a high voltage to PA5, we need to write a 1 to bit 5 of the output register:

```
volatile unsigned int* gpioa_odr = (unsigned int*) 0x48000014;  
*gpioa_odr = 0x00000020; // ODR5 = 1 -> PA5 high
```

3.8.5. The Bit Set/Reset Register

The STM32 also provides a second way to write to the output register: the bit set/reset register, `GPIOx_BSRR`. This register is split into two halves. Bits 15:0 (BS) set the corresponding output bit to 1, and bits 31:16 (BR) reset the corresponding output bit to 0. Writes to this register are atomic, meaning individual bits can be safely set or cleared without disturbing an interrupt that might be modifying other bits in `ODR` at the same time,

3.8. Example Application: Blink an LED

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A to F)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16: Reserved, must be kept at reset value.

Bits 15:0 **ODR[15:0]**: Port output data I/O pin y (y = 15 to 0)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and/or reset by writing to the GPIOx_BSRR register (x = A..F).

Figure 3.18.: The GPIO port output data register (GPIOx_ODR) sets the voltage of each output pin

something that isn't guaranteed when reading, modifying, and writing back the entire ODR register directly. We'll make use of BSRR again once we introduce interrupts later in this book.

3.8.6. Accessing Registers

Now that we know which registers to use, how do we access them in C? Just as with any other memory-mapped location, we create a pointer to the register's address, then dereference that pointer to read or write its value.

```
uint32_t* reg_p = (uint32_t*) 0x48000000;  
*reg_p = 0x00000005;
```

3. General Purpose Input/Output (GPIO)

8.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to F)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BR[15:0]**: Port x reset I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BS[15:0]**: Port x set I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

Figure 3.19.: The GPIO bit set/reset register (GPIOx_ODR) provides atomic writes to set or reset a GPIO output pin.

3.8. Example Application: Blink an LED

3.8.7. Finding the Register Address

In the reference manual, each register lists an **address offset**, indicating how far that register is from the GPIO port's **base address**. For example, GPIOx_ODR has an address offset of 0x14.

To find the base address for a specific port, we consult the peripheral boundary address table in the reference manual, shown in Figure 3.20.

Table 1. STM32F0xx peripheral register boundary addresses (continued)

Bus	Boundary address	Size	Peripheral	Peripheral register map
AHB2	0x4800 1400 - 0x4800 17FF	1KB	GPIOF	Section 8.4.12 on page 164
	0x4800 1000 - 0x4800 13FF	1KB	GPIOE	Section 8.4.12 on page 164
	0x4800 0C00 - 0x4800 0FFF	1KB	GPIOD	Section 8.4.12 on page 164
	0x4800 0800 - 0x4800 0BFF	1KB	GPIOC	Section 8.4.12 on page 164
	0x4800 0400 - 0x4800 07FF	1KB	GPIOB	Section 8.4.12 on page 164
	0x4800 0000 - 0x4800 03FF	1KB	GPIOA	Section 8.4.12 on page 164
-	0x4002 4400 - 0x47FF FFFF	~128 MB	Reserved	-
AHB1	0x4002 4000 - 0x4002 43FF	1 KB	TSC	Section 16.6.11 on page 326
	0x4002 3400 - 0x4002 3FFF	3 KB	Reserved	-
	0x4002 3000 - 0x4002 33FF	1 KB	CRC	Section 12.5.6 on page 232
	0x4002 2400 - 0x4002 2FFF	3 KB	Reserved	-
	0x4002 2000 - 0x4002 23FF	1 KB	FLASH interface	Section 3.5.9 on page 75
	0x4002 1400 - 0x4002 1FFF	3 KB	Reserved	-
	0x4002 1000 - 0x4002 13FF	1 KB	RCC	Section 6.4.15 on page 136
	0x4002 0800 - 0x4002 0FFF	2 KB	Reserved	-
	0x4002 0400 - 0x4002 07FF	1 KB	DMA2	Section 10.6.8 on page 212
	0x4002 0000 - 0x4002 03FF	1 KB	DMA	Section 10.6.8 on page 212
-	0x4001 8000 - 0x4001 FFFF	32 KB	Reserved	-

Figure 3.20.: Peripheral boundary addresses from the reference manual.

Each GPIO port occupies its own 1 KB block of address space

GPIOA's base address is 0x4800 0000. Adding the ODR offset gives us the address of GPIOA's output register:

3. General Purpose Input/Output (GPIO)

$$0x4800\ 0000 + 0x14 = 0x4800\ 0014$$

3.8.8. The “volatile” Keyword

Notice that all our register pointers have been declared with the `volatile` keyword. The `volatile` keyword is a type qualifier that forces the compiler to generate an instruction for every access to that variable. The compiler’s optimizer will often notice that some lines of embedded code appear to “do nothing” from an algorithmic perspective (nothing ever reads the variable we just wrote, as far as the compiler can tell) and will optimize those instructions out of the program entirely. Since we know these writes affect physical hardware, and not just program state, the `volatile` keyword prevents the compiler from making this mistake.

3.8.9. Putting it Together

Combining everything we’ve covered, here is a complete program that blinks the green LED on the NUCLEO-F091RC:

```
int main(void)
{
    // ---- Enable GPIOA clock ----
    volatile unsigned int* rcc_ahbenr = (unsigned int*)0x40021014;
    *rcc_ahbenr = 0x00020000; // Enable GPIOA (IOPAEN)

    // ---- GPIOA registers ----
    volatile unsigned int* gpioa_moder = (unsigned int*)0x48000000;
    volatile unsigned int* gpioa_odr  = (unsigned int*)0x48000014;

    // Set PA5 as output
    // MODER5 = 01 -> bits [11:10]
```

3.9. Example Application: Read a Button Press

```
*gpioa_moder = 0x00000400;

volatile unsigned int i;
while (1)
{
    // LED ON (PA5 = 1)
    *gpioa_oder = 0x00000020;
    for (i = 0; i < 100000; i++) { }

    // LED OFF (PA5 = 0)
    *gpioa_oder = 0x00000000;
    for (i = 0; i < 100000; i++) { }
}
```

This program will run indefinitely, blinking the LED on and off, until power is removed from the board. The `while(1)` loop is an infinite loop, and is found in nearly all embedded systems programs; we want our application to keep running so long as the device is powered, rather than terminate.

3.9. Example Application: Read a Button Press

Let's apply what we've learned to a second example: reading the state of the on-board user button (B1) and using it to control the LED.

3.9.1. Reading a Button Press with GPIO

Just as with the LED, we first need to identify which pin the button is connected to. Consulting the development board schematic, shown in Figure 3.21, we find that the button is connected to PC13. Note also

3. General Purpose Input/Output (GPIO)

that these buttons are “normally open,” meaning the switch is open (not connected) when the button is not being pressed.

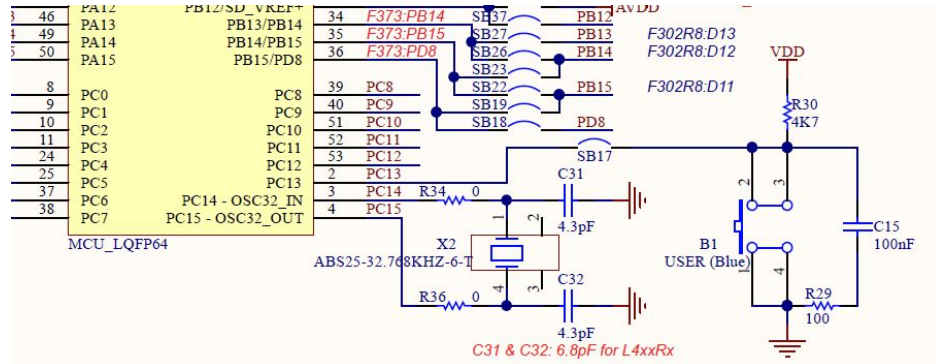


Figure 3.21.: The NUCLEO-F091RC’s user button (B1) is connected to PC13

3.9.2. Configuring the GPIO as an Input

In order to read a voltage on a GPIO pin, we must:

- Set the pin as an input (mode register)
- Configure a pull-up resistor, if one doesn’t already exist externally (resistor enable register)
- Read the value of the pin (input data register)

Setting PC13 as an input requires writing 00 to its corresponding two-bit field in `GPIOC_MODER` (bits 27:26):

```
volatile unsigned int* gpioc_moder = (unsigned int*) 0x48000800;  
*gpioc_moder = 0x00000000; // MODER13 = 00 -> bits [27:26]
```

3.9. Example Application: Read a Button Press

3.9.3. Reading a Voltage

The input data register, `GPIOx_IDR`, is used to read a voltage. If the voltage on an input pin is high, a 1 will appear in the corresponding bit position of this register; if the voltage is low, a 0 will appear.

3.9.4. Do We Need a Pull-Up Resistor?

If we look again at the schematic in Figure 3.21, we can see the board already includes a 4.7 k Ω pull-up resistor connected to the button. This means the pin will read a high voltage whenever the button isn't pressed, and a low voltage when it is pressed, without our needing to configure an internal pull-up resistor. Figure 3.22 highlights this resistor on the schematic.

8.4.5 GPIO port input data register (`GPIOx_IDR`) ($x = A$ to F)

Address offset: 0x10

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDR[15:0]**: Port x input data I/O pin y ($y = 15$ to 0)

These bits are read-only. They contain the input value of the corresponding I/O port.

Figure 3.22.: The NUCLEO-F091RC already includes an external pull-up resistor (R30) on the user button circuit

3. General Purpose Input/Output (GPIO)

3.9.5. Internal Pull-Up and Pull-Down Resistors

It's worth noting, however, that many circuits don't have the luxury of an external pull-up or pull-down resistor already in place. Because these resistors are so commonly needed (any switch or open-drain output requires one), most GPIO peripherals, including the STM32's, allow us to configure an *internal* resistor instead, saving us from having to add external hardware. Figure 3.23 shows a simplified illustration of how this internal resistor is wired into the GPIO hardware.

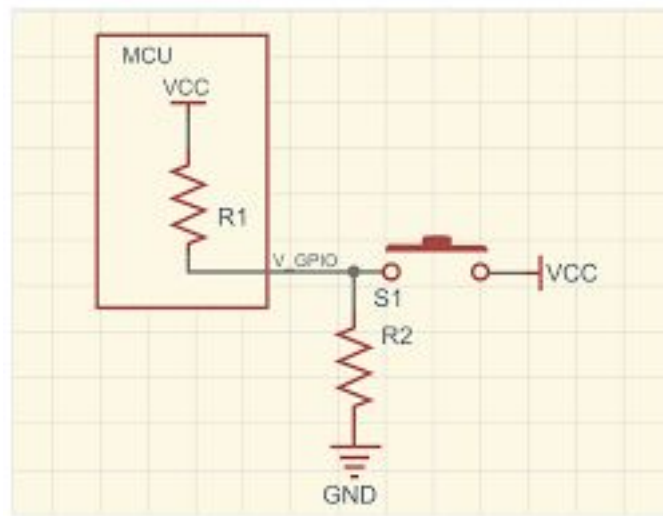


Figure 3.23.: Most GPIO peripherals include an internal pull-up or pull-down resistor that can be enabled in software, avoiding the need for external hardware

This internal resistor is configured using the `GPIOx_PUPDR` register, shown in Figure 3.24. Just like the mode register, two bits are used to configure each pin:

- 00: No pull-up or pull-down

3.9. Example Application: Read a Button Press

- 01: Pull-up
- 10: Pull-down
- 11: Reserved

8.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to F)

Address offset: 0x0C

Reset value: 0x2400 0000 (for port A)

Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **PUPDR[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

Figure 3.24.: The GPIO port pull-up/pull-down register (GPIOx_PUPDR) configures an internal pull resistor for each pin

Since our button already has an external pull-up, we'll leave PUPDR at its reset value (00, no internal resistor) for this example.

3.9.6. Putting it Together

Here's a complete program that reads the state of the button on PC13 and turns the LED on PA5 on when it is pressed:

3. General Purpose Input/Output (GPIO)

```
int main(void){
    // Enable GPIOA + GPIOC clocks (RCC_AHBENR)
    volatile unsigned int* rcc_ahbenr = (unsigned int*)0x40021014;
    *rcc_ahbenr = 0x00080000 | 0x00020000; // IOPCEN (bit 19) + IOPAEN (bit 18)

    // GPIOA registers (LED on PA5)
    volatile unsigned int* gpioa_moder = (unsigned int*)0x48000000;
    volatile unsigned int* gpioa_odr  = (unsigned int*)0x48000014;

    // GPIOC registers (Button on PC13)
    volatile unsigned int* gpioc_moder = (unsigned int*)0x48000800;
    volatile unsigned int* gpioc_idr  = (unsigned int*)0x48000810;

    // ---- Configure PA5 as output (MODER5 = 01 -> bits [11:10]) ----
    *gpioa_moder = 0x00000400;

    // ---- Configure PC13 as input (MODER13 = 00 -> bits [27:26]) ----
    *gpioc_moder = 0x00000000;

    // Initialize LED OFF (PA5 = 0)
    *gpioa_odr = 0x00000000;

    while(1){
        // If the button reads 0, turn the LED on.
        if(*gpioc_idr == 0x00000000){
            *gpioa_odr = 0x00000020; // PA5 high
        } else {
            *gpioa_odr = 0x00000000; // PA5 low
        }
    }
}
```

This code will run until power is removed, constantly checking the button

and setting the LED accordingly.

3.10. GPIO Hardware

Let's take a moment to observe how the control registers we've discussed create the hardware behavior we've described. Figure 3.25 shows a simplified schematic of the internals of the GPIO peripheral hardware.

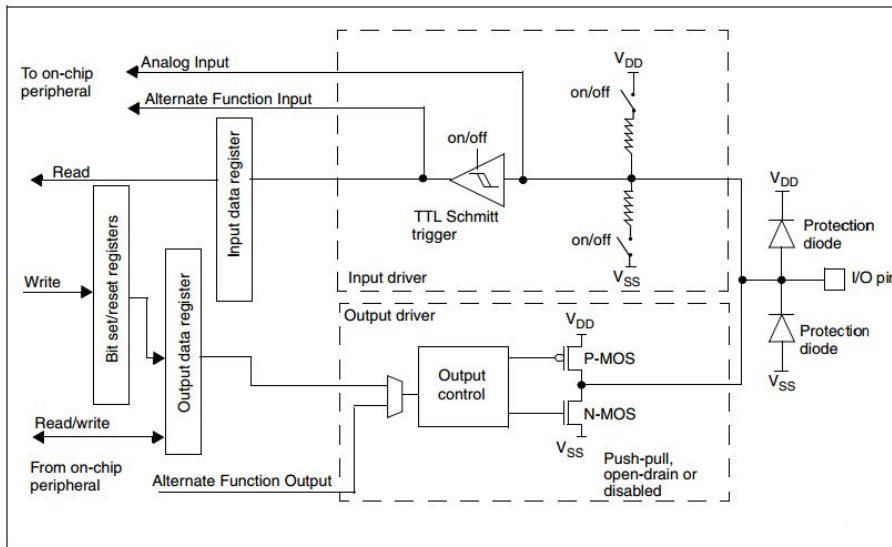


Figure 3.25.: Simplified internal schematic of the GPIO peripheral hardware, showing the input driver, output driver, and their associated control registers

On the output side, the bit set/reset registers and output data register feed into an output control block, which drives the pin using a push-pull or open-drain configuration, depending on how the pin has been configured.

3. General Purpose Input/Output (GPIO)

On the input side, a Schmitt trigger reads the voltage on the pin and stores a 1 or 0 into the input data register, which our program can then read.

3.11. Register Macros

Using pointers, we can write to any register we need and have total control over the hardware. However, it's tedious to create a pointer and dereference it every time we want to access a peripheral. We can condense this using preprocessor macros.

Preprocessor macros are lines of code that begin with a “#” symbol. These lines get replaced with actual C syntax by the preprocessor before the compiler runs, according to what the macro defines. We can use the `#define` directive to create macros that expand into dereferenced pointers:

```
#define GPIOA_MODER *((unsigned int*) 0x48000000)
#define GPIOA_ODR   *((unsigned int*) 0x48000014)
#define GPIOC_MODER *((unsigned int*) 0x48000800)
#define GPIOC_IDR   *((unsigned int*) 0x48000810)
#define RCC_AHBENR  *((unsigned int*) 0x40021014)
```

The preprocessor will replace every instance of `GPIOA_ODR` in our code with `*((unsigned int*) 0x48000014)`, letting us both read and write the register just by using its name. Let's update our button/LED example to use these macros:

```
#define RCC_AHBENR  *((unsigned int*) 0x40021014)
#define GPIOA_MODER *((unsigned int*) 0x48000000)
#define GPIOA_ODR   *((unsigned int*) 0x48000014)
#define GPIOC_MODER *((unsigned int*) 0x48000800)
#define GPIOC_IDR   *((unsigned int*) 0x48000810)
```

3.11. Register Macros

```
int main(void){
    // Enable GPIOA (bit 17) and GPIOC (bit 19) clocks
    RCC_AHBENR = 0x00020000 | 0x00080000;

    // ---- Configure PA5 as output ----
    GPIOA_MODER = 0x00000400;

    // ---- Configure PC13 as input ----
    GPIOC_MODER = 0x00000000;

    // Turn LED OFF initially
    GPIOA_ODR = 0x00000000;

    while(1){
        if(GPIOC_IDR == 0x00000000){
            GPIOA_ODR = 0x00000020; // PA5 = 1 (LED ON)
        } else {
            GPIOA_ODR = 0x00000000; // PA5 = 0 (LED OFF)
        }
    }
}
```

Manufacturers understand this need quite well, and to make their products more attractive to programmers, they typically provide a header file with these macros already defined for every register on the chip. For the STM32, this comes in the form of a CMSIS-style device header, `stm32f0xx.h`, which groups each peripheral's registers into a C struct, accessed through a pointer named after the peripheral:

```
#include "stm32f0xx.h"

int main(void){
```

3. General Purpose Input/Output (GPIO)

```
// Enable GPIOA (bit 17) and GPIOC (bit 19) clocks
RCC->AHBENR = 0x00020000 | 0x00080000;

// ---- Configure PA5 as output ----
GPIOA->MODER = 0x00000400;

// ---- Configure PC13 as input ----
GPIOC->MODER = 0x00000000;

// Turn LED OFF initially
GPIOA->ODR = 0x00000000;

while(1){
    if(GPIOC->IDR == 0x00000000){
        GPIOA->ODR = 0x00000020; // PA5 high
    } else {
        GPIOA->ODR = 0x00000000; // PA5 low
    }
}
```

By including "stm32f0xx.h", we gain access to GPIOA, GPIOC, and RCC, each already pointing to the correct base address for our target device, along with named fields (MODER, ODR, IDR, etc.) for every register they contain.

3.12. Bit Masking

When manipulating individual bits in a control register, one of our biggest concerns is being able to read from and write to individual bits without disturbing the others. Since memory is divided into byte-addressable (or larger) locations, we can't read or write a single bit in isolation; we always

3.12. Bit Masking

have to operate on the entire register. To isolate a single bit (or a handful of bits), we use a **bit mask**.

3.12.1. Understanding the Problem

Consider our earlier example, where we checked the entire input register to see if the button was pressed:

```
if(GPIOC->IDR == 0x00000000){ /* ... */ }
```

This only works because PC13 is the only pin we care about on GPIOC. If another input, say a second button on PC2, were also being read, and it happened to be high, `GPIOC->IDR` would no longer equal `0x00000000` even when our button of interest was pressed. We need a way to check the state of bit 13 alone, ignoring every other bit in the register.

3.12.2. Creating a Bit Mask

A bit mask is a binary value constructed so that, when combined with another value using a bitwise operator, it isolates, sets, or clears specific bits while leaving the rest untouched. Figure 3.26 illustrates how a bitwise AND operation with a mask isolates a single bit, zeroing out every other bit in the result.

To check only bit 13 of `GPIOC->IDR`, we construct a mask with a single 1 in the bit 13 position (`0x00002000`) and apply it with a bitwise AND:

```
if((GPIOC->IDR & 0x00002000) == 0x00000000){  
    // button is pressed  
}
```

3. General Purpose Input/Output (GPIO)

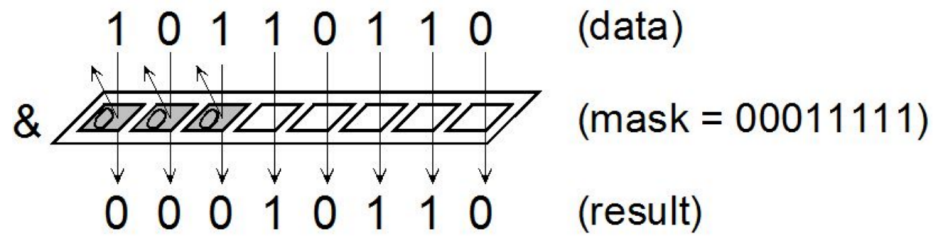


Figure 3.26.: A bitwise AND with a mask isolates the bits of interest, zeroing out every other bit

3.12.3. Bit Masking for Setting and Clearing Bits

We can use a similar approach to set or clear individual bits without disturbing the rest of a register. To *set* a bit, we use the bitwise OR operator (`|`) with a mask containing a 1 in the position we want to set:

```
GPIOA->ODR = GPIOA->ODR | 0x00000020; // set PA5, leave other bits alone
```

To *clear* a bit, we use the bitwise AND operator with the *inverse* of our mask, which places a 0 in the position we want to clear and 1's everywhere else:

```
GPIOA->ODR = GPIOA->ODR & ~(0x00000020); // clear PA5, leave other bits alone
```

C provides shorthand assignment operators for both of these operations:

```
GPIOA->ODR |= 0x00000020; // equivalent to: GPIOA->ODR = GPIOA->ODR | 0x00000020;  
GPIOA->ODR &= ~(0x00000020); // equivalent to: GPIOA->ODR = GPIOA->ODR & ~(0x00000020);
```


3.12.4. Creating Readable Masks with the Shift Operator

Hand-writing hex masks like 0x00000020 makes it difficult to tell, at a glance, which bit is being targeted. We can improve readability by using the left-shift operator (<<) to construct the mask instead:

```
GPIOA->ODR |= (1u << 5);    // set PA5
GPIOA->ODR &= ~(1u << 5);    // clear PA5
```

The `1u << 5` expression takes the value 1 and shifts it left five times, landing a single 1 bit in position 5. This is functionally identical to writing 0x00000020, but immediately communicates which pin we're targeting. Most compilers will recognize this as a constant expression and generate the same code as if we had hardcoded the value; we get the readability improvement without any runtime cost.

3.12.5. Masking Registers with Paired Values

Recall that some STM32 registers, like `MODER` and `PUPDR`, use two bits per pin rather than one. This introduces a subtlety that we didn't encounter with single-bit registers like `ODR`. Suppose we want to set PC4 as an output by writing 01 to bits 9:8 of `GPIOC->MODER`:

```
GPIOC->MODER |= (0x1u << 8); // NOT guaranteed to work correctly!
```

Using only an OR here is *not* safe. If bit 9 already happened to contain a 1 (perhaps left over from a different mode), the OR operation would leave it set, resulting in an unintended mode such as 11 (analog) rather than the 01 (output) we wanted. Since OR can only set bits, never clear them, we must first clear both bits in the field before setting the ones we want:

3. General Purpose Input/Output (GPIO)

```
GPIOC->MODER &= ~(0x3u << (4 * 2)); // clear bits [9:8]
GPIOC->MODER |= (0x1u << (4 * 2)); // set to 01 (output)
```

Here, 0x3u (0b11) is used as the clearing mask, since it spans both bits of the two-bit field, and $4 * 2$ computes the starting bit position for pin 4's field (each pin occupies 2 bits, so pin N 's field begins at bit $N * 2$). This two-step clear-then-set pattern is required any time we're working with a multi-bit field, and you'll use it constantly when configuring MODER and PUPDR.

3.13. Additional Notes

A few practical considerations are worth keeping in mind as you begin writing your own GPIO applications.

3.13.1. Polling for Inputs

In our button example, our `while(1)` loop constantly re-checks the state of the button on every iteration. This technique is called **polling**. Polling is simple to implement, but it should only be used for inputs that don't need to be responded to immediately or that don't change very frequently. If we need to respond to an input the instant it changes, or if checking that input is too costly to do constantly, we'll need to use an **interrupt** instead, which we will cover in a later chapter.

3.13.2. Button Debouncing

When a mechanical button is pressed, the signal often “bounces” between high and low several times over a very short period before settling, due to the physical contacts inside the switch. If read naively, this can cause our

program to register several presses instead of one. A short delay inserted after detecting the first change in the button's state is a simple way to filter out this bouncing; the appropriate delay length varies by switch and is often determined empirically.

3.14. Recap

In this chapter we've learned:

- How peripherals let a microcontroller interface with the outside world
- Where to find the documentation for a development board, a microcontroller family, and a specific microcontroller
- How to control peripherals by writing to memory-mapped control registers
- How to configure and use the STM32 GPIO peripheral's `MODER`, `ODR`, `IDR`, `PUPDR`, and `BSRR` registers
- Why STM32 peripherals require their clock to be enabled through the `RCC` before they can be used
- How to use pointers, preprocessor macros, and the vendor-provided device header to access control registers
- How to use bit masking to isolate, set, and clear individual bits (or multi-bit fields) without disturbing the rest of a register

4. Embedded Programming

i Draft status

This chapter has not yet been written. It will cover functionalizing code, hardware abstraction layers, software state machines, clock sources, and interrupts on STM32.

Part III.

Peripherals

5. Timers

i Draft status

This chapter has not yet been written. It will cover counters, timers, and the STM32 timer peripherals.

6. Analog-to-Digital Conversion (ADC)

i Draft status

This chapter has not yet been written. It will cover the STM32's analog-to-digital converter(s), configuring the ADC, sampling analog inputs, and reading conversion results.

7. Serial Communication

i Draft status

This chapter has not yet been written. It will cover serial communication peripherals on the STM32, such as USART, and how to use them to communicate with other devices.

